

Kiến trúc Định tuyến Động: Tối ưu hóa Chi phí – Hiệu năng trong Hệ thống AI Gia sư thông qua Lựa chọn Chiến lược Truy xuất Thích nghi

Tác giả: Nguyễn Văn Đăng

Đơn vị: Học viện Công nghệ Bưu Chính Viễn Thông

Liên hệ: nguyenvandang22012k5@gmail.com

Tóm tắt

Các hệ thống AI Gia sư hiện nay phần lớn áp dụng một chiến lược truy xuất tri thức cố định cho toàn bộ truy vấn đầu vào-hoặc chỉ lấy các đoạn văn bản liên quan-RAG, hoặc đưa toàn bộ phần tài liệu thuộc lĩnh vực liên quan của truy vấn vào cửa sổ ngữ cảnh của mô hình-LCP. Cách tiếp cận tĩnh này tạo ra một mâu thuẫn cơ bản: lãng phí tài nguyên tính toán khi áp dụng LCP theo lĩnh vực cho các truy vấn đơn giản, hoặc suy giảm chất lượng phản hồi khi ép RAG xử lý các câu hỏi đòi hỏi suy luận tổng hợp.

Bài nghiên cứu này đề xuất Kiến trúc Định tuyến Động (Dynamic Routing Architecture - DRA), một cơ chế cho phép hệ thống tự động phân tích đặc tính của từng truy vấn và lựa chọn chiến lược xử lý phù hợp tại thời gian thực. Bài toán được hình thức hóa dưới dạng tối ưu hóa có ràng buộc, trong đó mục tiêu là cực tiểu hóa chi phí tổng hợp trong khi vẫn duy trì chất lượng sự phạm theo ngưỡng mục tiêu θ . Nghiên cứu trình bày khung phương pháp, thiết kế kiến trúc, benchmark 1.008 truy vấn và kết quả thực nghiệm định lượng sau khi hoàn tất pipeline đánh giá. Trên tập test 336 truy vấn, DRA-DistilBERT đạt RAGAS 0,828 với chi phí 3,32 USD/1K truy vấn, giảm 65,6% chi phí so với Pure-LCP trong khi vẫn giữ 99,2% chất lượng RAGAS của Pure-LCP.

Từ khóa: Định tuyến động, Retrieval-Augmented Generation, Long-Context Prompting, AI Gia sư, Tối ưu hóa chi phí, Tối ưu hóa có ràng buộc.

1. Giới thiệu

1.1 Bối cảnh và Động lực Nghiên cứu

Sự xuất hiện và phổ biến nhanh chóng của các mô hình ngôn ngữ lớn trong thập kỷ gần đây đã mở ra một chương mới cho lĩnh vực giáo dục hỗ trợ bởi trí tuệ nhân tạo. Không còn bị giới hạn bởi các kịch bản hỏi-đáp cứng nhắc hay cơ sở tri thức tĩnh, các hệ thống AI Gia sư thế hệ mới có khả năng giải thích khái niệm, phân tích lập luận, và phản hồi các câu hỏi mở với mức độ linh hoạt chưa từng có. Tuy nhiên, để khai thác tiềm năng này trong thực tế-đặc biệt khi hệ thống cần trả lời chính xác dựa trên một nguồn tài liệu học thuật cụ thể-một thách thức kỹ thuật căn bản vẫn còn tồn tại: làm thế nào để cung cấp cho LLM đúng lượng ngữ cảnh tri thức cần thiết, không thừa cũng không thiếu, cho từng câu hỏi cụ thể?

Để làm rõ bản chất của thách thức này, hãy xem xét hai câu hỏi tiêu biểu từ môi trường học thuật đại học:

- Câu hỏi A: "What does the z-score measure in the standard normal distribution?"
- Câu hỏi B: "How does monopoly differ from perfect competition in pricing and output decisions?"

Câu hỏi A là một truy vấn tra cứu điểm: câu trả lời nằm gọn trong phần giải thích về z-score trong chương The Normal Distribution của giáo trình Statistics, có thể được trích xuất trực tiếp từ một vị trí cụ thể trong tài liệu. Câu hỏi B, ngược lại, đòi hỏi hệ thống đối chiếu hai cấu trúc thị trường được trình bày ở các chương khác nhau của giáo trình Microeconomics, nhận diện các khác biệt về quyền lực thị trường, quyết định giá và sản lượng, rồi tổng hợp thành một lập luận mạch lạc-một quá trình suy luận đa nguồn không thể rút gọn về một thao tác tra cứu đơn lẻ.

Hai câu hỏi này đại diện cho hai cực đối lập trên phổ độ phức tạp của truy vấn giáo dục. Thế nhưng, đa số các hệ thống AI Gia sư hiện tại lại xử lý cả hai theo cùng một phương thức cố định-áp dụng đồng nhất một chiến lược truy xuất tri thức cho toàn bộ đầu vào bất kể đặc tính của từng câu hỏi. Đây chính là khoảng trống thiết kế mà nghiên cứu này hướng đến giải quyết.

1.2 Hai Chiến lược Truy xuất Hiện có

Hiện tại, hai hướng tiếp cận chủ đạo đang được triển khai rộng rãi để cung cấp ngữ cảnh tri thức cho LLM trong các hệ thống hỏi-đáp dựa trên tài liệu.

Retrieval-Augmented Generation (RAG)

RAG hoạt động theo cơ chế tìm kiếm có chọn lọc. Khi nhận một truy vấn đầu vào, hệ thống trước tiên chuyển đổi câu hỏi thành một vector biểu diễn ngữ nghĩa dày đặc thông qua một mô hình nhúng. Vector này được sử dụng để tìm kiếm trong cơ sở tri thức-vốn đã được lập chỉ mục dưới dạng các đoạn văn bản với vector tương ứng-nhằm lấy ra một tập hợp nhỏ các đoạn có độ tương đồng ngữ nghĩa cao nhất. Chỉ những đoạn này được đưa vào prompt gửi đến LLM, giữ cho kích thước ngữ cảnh ở mức tối thiểu.

Phương pháp này mang lại hiệu quả kinh tế đáng kể: chi phí token thấp, tốc độ xử lý nhanh, và khả năng mở rộng tốt khi kích thước tài liệu tăng lên. Tuy nhiên, hiệu quả của RAG hoàn toàn phụ thuộc vào độ chính xác của bước truy xuất. Với các câu hỏi đòi hỏi suy luận tổng hợp trên nhiều nguồn-như câu hỏi B ở trên-bước tìm kiếm vector không thể đảm bảo lấy đủ tất cả các mảnh thông tin cần thiết, dẫn đến câu trả lời thiếu toàn diện hoặc xuất hiện hiện tượng ảo giác khi mô hình cố gắng lấp đầy các lỗ hổng ngữ cảnh bằng suy diễn không có căn cứ.

Long-Context Prompting (LCP)

LCP tiếp cận vấn đề theo chiều ngược lại: thay vì chọn lọc một vài đoạn ngắn, hệ thống đưa toàn bộ

phần tài liệu thuộc lĩnh vực đã xác định của truy vấn vào cửa sổ ngữ cảnh của LLM trong một lần xử lý duy nhất. Trong nghiên cứu này, LCP được hiểu nhất quán là Domain-scoped LCP: không đưa toàn bộ corpus của cả ba lĩnh vực vào mọi truy vấn, mà chỉ đưa phần corpus đã tiền xử lý của lĩnh vực liên quan. Kích thước ngữ cảnh theo lĩnh vực được đo bằng tokenizer của mô hình sử dụng trong thực nghiệm sau bước tiền xử lý. Sự ra đời của các mô hình ngữ cảnh dài thế hệ mới-như DeepSeek-V4-Pro với khả năng xử lý hơn một triệu token, hay Claude 3.5 với cửa sổ ngữ cảnh 200.000 token-đã biến thiết lập này từ một ý tưởng lý thuyết thành một lựa chọn kỹ thuật khả thi.

Ưu thế của LCP theo lĩnh vực là rõ ràng với các truy vấn tổng hợp phức tạp: mô hình được cung cấp toàn bộ ngữ cảnh trong phạm vi lĩnh vực liên quan, qua đó giảm đáng kể rủi ro không cung cấp đủ thông tin đầu vào. Tuy nhiên, điều này không đồng nghĩa với việc mô hình chắc chắn khai thác hoàn hảo mọi thông tin trong ngữ cảnh dài. LCP vẫn chịu ảnh hưởng của chi phí token, độ trễ prefill và hiện tượng lost-in-the-middle khi thông tin quan trọng nằm ở vùng giữa của cửa sổ ngữ cảnh. Vì vậy, LCP nên được xem là chiến lược tăng độ phủ ngữ cảnh với chi phí cao, không phải giải pháp bảo đảm tuyệt đối chất lượng.

1.3 Khoảng trống Nghiên cứu: Sự Đánh đổi Tính và Hệ quả của nó

Mâu thuẫn cốt lõi nằm ở chỗ: cả RAG lẫn LCP đều là những lựa chọn *tốt nhất* trong phạm vi ứng dụng phù hợp của mình, nhưng lại là những lựa chọn *tệ nhất* khi bị áp dụng sai phạm vi. Việc bắt buộc chọn một trong hai chiến lược và áp dụng đồng đều cho mọi truy vấn tất yếu dẫn đến hai hệ quả tiêu cực:

Over-provisioning xảy ra khi LCP được áp dụng cho các truy vấn tra cứu đơn giản-những câu hỏi mà RAG hoàn toàn có thể xử lý hiệu quả với chỉ một vài đoạn ngữ cảnh. Mỗi truy vấn loại này tiêu tốn một lượng token gấp nhiều lần mức cần thiết, tạo ra chi phí vận hành dư thừa tích lũy theo quy mô sử dụng.

Trong bối cảnh một nền tảng học trực tuyến với hàng triệu tương tác mỗi ngày, sự lãng phí này chuyển hóa trực tiếp thành chi phí tài chính không cần thiết và dấu chân tính toán có thể tránh được.

Under-performing xảy ra ở chiều đối lập: khi RAG được cưỡng ép xử lý các truy vấn tổng hợp đòi hỏi hiểu biết liên tài liệu. Trong những trường hợp này, bước truy xuất vector-dù được tối ưu tốt đến đâu-vẫn không đủ khả năng xác định và tập hợp đầy đủ tất cả các mảnh thông tin liên quan. Kết quả là những câu trả lời thiếu chiều sâu, bỏ qua các mối liên hệ quan trọng, hoặc tệ hơn, những câu trả lời nghe có vẻ mạch lạc nhưng thực chất chứa đựng các suy luận sai lệch được mô hình tự điền vào khoảng trống ngữ cảnh-điều đặc biệt nguy hại trong môi trường giáo dục, nơi tính chính xác và trung thực của thông tin có ý nghĩa sự phạm trực tiếp.

Tình trạng này cho thấy một khoảng trống thiết kế rõ ràng: không phải là sự yếu kém của bản thân RAG hay LCP, mà là sự thiếu vắng một lớp điều phối thông minh có khả năng phân bổ đúng chiến lược cho đúng loại truy vấn.

1.4 Đề xuất Nghiên cứu

Để lấp đầy khoảng trống này, bài nghiên cứu đề xuất Kiến trúc Định tuyến Động: một cơ chế phân tích đặc tính ngôn ngữ của từng truy vấn đầu vào và tự động lựa chọn chiến lược xử lý tối ưu giữa RAG và LCP tại thời gian thực, mà không yêu cầu điều chỉnh lại kiến trúc hay tham số của mô hình ngôn ngữ gốc.

Câu hỏi nghiên cứu trung tâm: Liệu có thể xây dựng một cơ chế định tuyến thông minh-có khả năng nhận diện đặc tính của từng truy vấn và lựa chọn chiến lược xử lý tối ưu tại thời gian thực-mà không yêu cầu điều chỉnh lại mô hình ngôn ngữ gốc?

DRA được hình thức hóa như một bài toán tối ưu hóa có ràng buộc, trong đó mục tiêu là cực tiểu hóa chi phí tổng hợp trong khi vẫn đảm bảo chất lượng phản hồi sự phạm đạt ngưỡng tối thiểu θ . Bài toán được

xấp xỉ thông qua một bộ phân loại học máy hạng nhẹ, có khả năng ra quyết định định tuyến trong vài mili-giây-đủ nhanh để triển khai trong luồng xử lý thời gian thực.

1.5 Đóng góp Chính của Nghiên cứu

Nghiên cứu này đóng góp ba thành phần có liên hệ chặt chẽ với nhau:

Về mặt hình thức hóa bài toán, chúng tôi cung cấp một khung tối ưu hóa có ràng buộc cho bài toán lựa chọn chiến lược truy xuất thích nghi trong hệ thống AI Gia sư-khung này phân tách rõ ràng ba chiều đánh giá, cho phép tùy biến ưu tiên theo nhu cầu triển khai cụ thể thông qua các trọng số w_1 , w_2 và ngưỡng θ .

Về mặt kiến trúc kỹ thuật, chúng tôi trình bày thiết kế chi tiết của DRA-một lớp middleware phi xâm lấn có thể tích hợp với bất kỳ LLM nào mà không cần fine-tune mô hình sinh câu trả lời, bao gồm đặc tả bộ trích xuất đặc trưng đa tầng 786 chiều và hai kiến trúc phân loại nhẹ với đặc tính hiệu năng khác nhau phù hợp cho các kịch bản triển khai khác nhau.

Về mặt thiết lập đánh giá và kết quả, chúng tôi xây dựng một benchmark giáo dục tổng hợp gồm 1.008 truy vấn trải rộng trên ba lĩnh vực tri thức, với 336 mẫu được gán nhãn thủ công bởi nhiều người chú thích. DRA được đánh giá theo ba chiều độc lập: chi phí token, độ trễ end-to-end và điểm RAGAS. Kết quả thực nghiệm cho thấy DRA-DistilBERT đạt điểm cân bằng chi phí-chất lượng tốt hơn các chiến lược tĩnh trong cấu hình $\theta = 0,7$: giảm đáng kể chi phí so với Pure-LCP trong khi cải thiện rõ chất lượng so với Pure-RAG.

1.6 Cấu trúc Bài báo

Phần còn lại của bài báo được tổ chức như sau. Mục 2 khảo sát các công trình liên quan theo ba nhánh: RAG và các biến thể, mô hình ngữ cảnh dài, và các phương pháp định tuyến thích nghi. Mục 3 hình thức

hóa bài toán tối ưu hóa. Mục 4 mô tả kiến trúc hệ thống DRA. Mục 5 trình bày thiết lập thực nghiệm. Mục 6 trình bày kết quả thực nghiệm và phân tích định lượng. Mục 7 thảo luận về giới hạn và hướng mở rộng. Mục 8 kết luận.

2. Các Công trình Liên quan

Nghiên cứu này nằm ở giao điểm của ba dòng nghiên cứu chính: tăng cường truy xuất cho mô hình ngôn ngữ, mở rộng khả năng xử lý ngữ cảnh dài, và định tuyến thích nghi trong các hệ thống AI. Phần này điểm qua các công trình tiêu biểu trong từng nhánh và phân tích cụ thể điểm khác biệt mà DRA mang lại.

2.1 Retrieval-Augmented Generation và các Biến thể

Nền tảng lý thuyết của RAG được đặt bởi Lewis và cộng sự (2020), trong công trình định nghĩa kiến trúc kết hợp giữa một bộ truy xuất dày đặc và một mô hình sinh văn bản. Đây là bước chuyển quan trọng so với các phương pháp truy xuất thưa thớt truyền thống như BM25, cho phép mô hình tìm kiếm dựa trên nghĩa thực sự của câu hỏi thay vì chỉ đối sánh từ khóa bề mặt.

Các hướng cải tiến RAG sau đó tập trung vào ba điểm nút kỹ thuật. Thứ nhất, về chiến lược phân đoạn tài liệu: Shi và cộng sự (2023) chỉ ra rằng kích thước đoạn văn bản có ảnh hưởng đáng kể đến chất lượng truy xuất-đoạn quá ngắn thiếu ngữ cảnh cục bộ, đoạn quá dài làm loãng tín hiệu ngữ nghĩa và làm giảm độ chính xác của bước so khớp vector. Thứ hai, về biểu diễn ngữ nghĩa dày đặc: Karpukhin và cộng sự (2020) đề xuất DPR, sử dụng cặp mô hình nhúng được huấn luyện theo cách thức đối lập để tối ưu trực tiếp mục tiêu truy xuất. Thứ ba, về tái xếp hạng kết quả: Nogueira và Cho (2019) đề xuất sử dụng BERT như một mô hình phân loại nhị phân để đánh giá lại mức độ liên quan của từng đoạn sau khi đã có top-k ứng viên ban đầu, giúp nâng cao độ chính xác của tập ngữ cảnh cuối cùng.

Mặc dù ba hướng cải tiến này đều mang lại tiến bộ đáng kể về chất lượng truy xuất, chúng đều ngầm giả định rằng RAG là chiến lược phù hợp cho mọi truy vấn. Câu hỏi *khi nào RAG thực sự là lựa chọn tốt nhất*-và khi nào không-vẫn còn bỏ ngỏ trong toàn bộ dòng nghiên cứu này. Đây là điểm khởi đầu của DRA.

2.2 Mô hình Ngữ cảnh Dài và Hiện tượng "Lost-in-the-Middle"

Sự ra đời của các mô hình ngôn ngữ có cửa sổ ngữ cảnh cực lớn-DeepSeek-V4-Pro (>1 triệu token), Claude 3.5 (200.000 token), GPT-4 Turbo (128.000 token)-đã làm thay đổi đáng kể cục diện kỹ thuật, biến LCP theo lĩnh vực từ một giải pháp bị ràng buộc bởi giới hạn phần cứng thành một lựa chọn khả thi trong nhiều tình huống thực tế.

Tuy nhiên, một phát hiện thực nghiệm quan trọng của Liu và cộng sự (2023) đặt ra câu hỏi về giá trị thực sự của khả năng này: các LLM hiện tại biểu hiện hiệu ứng "lost-in-the-middle"-xu hướng hệ thống trong việc ưu tiên xử lý thông tin nằm ở đầu và cuối cửa sổ ngữ cảnh, trong khi bỏ sót hoặc xử lý kém hơn thông tin nằm ở vùng giữa. Hiệu ứng này trở nên đặc biệt rõ rệt khi độ dài ngữ cảnh vượt quá 10.000–20.000 token, tức là ngay trong phạm vi kích thước tài liệu giáo dục điển hình.

Hệ quả của phát hiện này đối với nghiên cứu hiện tại là có tính hai chiều. Một mặt, nó xác nhận rằng LCP không phải là giải pháp toàn năng ngay cả khi chi phí không phải là ràng buộc-hiệu quả thực tế của LCP phụ thuộc vào vị trí thông tin liên quan trong cửa sổ ngữ cảnh, một yếu tố khó kiểm soát khi đưa toàn bộ phần tài liệu theo lĩnh vực vào. Mặt khác, phát hiện này ngầm chỉ ra rằng tồn tại một lớp truy vấn-cụ thể là những câu hỏi mà thông tin liên quan nằm rải rác và phân tán trong cùng lĩnh vực-mà LCP theo lĩnh vực có thể có lợi thế về khả năng tổng hợp so với RAG, kể cả khi xét đến hiệu ứng lost-in-the-middle. DRA được thiết kế để khai thác sự phân tầng này: chỉ định tuyến sang LCP khi độ phức tạp của truy vấn đủ

cao để xứng đáng với cả chi phí token lẫn chi phí khai thác ngữ cảnh dài đi kèm.

2.3 Định tuyến và Truy xuất Thích nghi

Đây là nhánh nghiên cứu có liên hệ trực tiếp nhất với đề xuất trong bài báo này, và cũng là nhánh đang phát triển nhanh nhất trong thời gian gần đây.

Adaptive-RAG (Jeong et al., 2024) là công trình gần nhất về tinh thần với DRA. Nghiên cứu đề xuất phân loại câu hỏi thành ba mức độ phức tạp và lựa chọn chiến lược truy xuất tương ứng cho mỗi mức, thay vì chỉ có hai lựa chọn nhị phân. Đây là bước tiến quan trọng so với các hệ thống đơn chiến lược, và là baseline cạnh tranh trực tiếp cần được đưa vào kế hoạch thực nghiệm hoàn chỉnh. Tuy nhiên, Adaptive-RAG có hai hạn chế đáng chú ý trong ngữ cảnh của bài toán này: (1) công trình không đề cập đến việc mô hình hóa tường minh sự đánh đổi chi phí token-tức là không tích hợp chi phí vào tiêu chí ra quyết định; và (2) framework không được thiết kế riêng cho bối cảnh giáo dục, nơi ngưỡng chất lượng sự phạm tối thiểu cần được xem như một ràng buộc vận hành rõ ràng chứ không chỉ là một mục tiêu mềm.

Self-RAG (Asai et al., 2023) đại diện cho một hướng tiếp cận khác về mặt triết lý: thay vì sử dụng một bộ phân loại bên ngoài, LLM tự học cách quyết định *khi nào* cần truy xuất thông qua việc sinh ra các *reflection tokens* đặc biệt trong quá trình inference. Đây là một ý tưởng thanh lịch về mặt thiết kế, vì nó tích hợp khả năng định tuyến trực tiếp vào bản thân

mô hình, loại bỏ độ trễ của bộ phân loại ngoài. Tuy nhiên, điều kiện tiên quyết của Self-RAG là mô hình ngôn ngữ gốc phải được fine-tune để nhận diện và sinh ra các reflection tokens này-một bước đòi hỏi tài nguyên tính toán đáng kể và làm phát sinh sự phụ thuộc vào một phiên bản mô hình cụ thể. DRA tránh hoàn toàn yêu cầu này bằng cách duy trì LLM gốc nguyên vẹn và đặt toàn bộ logic định tuyến ở lớp middleware bên ngoài.

RouteLLM (Ong et al., 2024) giải quyết một bài toán định tuyến liên quan nhưng về bản chất khác biệt: thay vì lựa chọn *chiến lược truy xuất*, RouteLLM lựa chọn *mô hình ngôn ngữ* để xử lý-định tuyến các truy vấn đơn giản sang mô hình nhẹ, rẻ hơn và các truy vấn phức tạp sang mô hình mạnh, đắt hơn. Đây là một hướng tối ưu chi phí có giá trị thực tiễn, nhưng không đề cập đến bài toán lựa chọn chiến lược truy xuất-hai bài toán này về nguyên tắc là độc lập và có thể được áp dụng đồng thời. DRA tập trung vào chiều truy xuất, không phải chiều lựa chọn mô hình.

CacheBlend (Yao et al., 2024) đề xuất tối ưu hóa chi phí tính toán của LCP thông qua cơ chế tái sử dụng KV-cache cho các đoạn tài liệu không thay đổi giữa các lần truy vấn liên tiếp. Đây là một cải tiến kỹ thuật thực sự hữu ích cho phía LCP, nhưng không giải quyết câu hỏi cơ bản hơn: liệu một truy vấn cụ thể có *cần* LCP ngay từ đầu hay không.

2.4 Tổng hợp và Định vị Đóng góp

Bảng 1. So sánh các công trình liên quan với DRA.

Công trình	Ý tưởng cốt lõi	Yêu cầu fine-tuning LLM gốc	Mô hình hóa chi phí	Áp dụng cho giáo dục	Định tuyến RAG ↔ LCP
Adaptive-RAG	Phân loại 3 mức độ phức tạp	Không	Không	Không	Một phần
Self-RAG	LLM tự quyết định truy xuất	Có	Không	Không	Không
RouteLLM	Định tuyến mô hình	Không	Có	Không	Không
CacheBlend	Tối ưu KV-cache cho LCP	Không	Một phần	Không	Không
DRA (đề xuất)	Định tuyến RAG↔LCP theo lĩnh vực có ràng buộc chất lượng	Không (chỉ fine-tune router nếu dùng DistilBERT)	Có	Có	Có

Điểm khác biệt cốt lõi của DRA nằm ở sự kết hợp của ba yêu cầu mà chưa có công trình nào giải quyết đồng thời: (1) định tuyến tường minh giữa RAG và LCP như hai cực của không gian chiến lược truy xuất; (2) hình thức hóa bài toán như tối ưu hóa có ràng buộc, tích hợp chi phí token và chất lượng vào cùng một khung ra quyết định thống nhất; và (3) áp dụng trong bối cảnh AI Gia sư với đặc thù sự phạm-nơi ngưỡng chất lượng tối thiểu θ không chỉ là tham số kỹ thuật mà còn mang ý nghĩa giáo dục trực tiếp.

Ngoài ra, không giống Self-RAG, DRA không yêu cầu bất kỳ thay đổi nào đối với mô hình ngôn ngữ nền tảng sinh câu trả lời. Điều này không loại trừ việc fine-tune một router/classifier nhỏ bên ngoài LLM. Kiến trúc middleware phi xâm lấn này cho phép DRA được tích hợp với GPT-4o, Claude, Gemini hay bất kỳ LLM nào khác mà không cần tái huấn luyện mô hình sinh-một thuộc tính quan trọng cho khả năng ứng dụng thực tế và tính linh hoạt trong vòng đời triển khai.

3. Định nghĩa Bài toán

Phần này hình thức hóa bài toán lựa chọn chiến lược truy xuất thích nghi dưới dạng một bài toán tối ưu hóa có ràng buộc. Chúng tôi lần lượt định nghĩa không gian hành động, các hàm mục tiêu và ràng buộc, phát biểu bài toán tối ưu tổng quát, rồi phân tích lý do tại sao bài toán này không thể giải trực tiếp và cần được tiếp cận thông qua xấp xỉ học máy.

3.1 Không gian Hành động

Cho Q là tập hợp tất cả các truy vấn đầu vào mà hệ thống AI Gia sư có thể nhận được. Mỗi phần tử $q \in Q$ là một câu hỏi tự nhiên được học viên đặt ra trong ngữ cảnh học tập, có thể thuộc bất kỳ dạng nào: tra cứu sự kiện, định nghĩa khái niệm, so sánh, hay tổng hợp đa nguồn.

Với mỗi truy vấn $q \in Q$, hệ thống thực hiện một quyết định nhị phân bằng cách ánh xạ q sang không gian hành động $A = \{0, 1\}$:

$$a = 0 \Rightarrow \text{Định tuyến sang luồng RAG}$$

Ưu tiên chi phí-phù hợp với truy vấn tra cứu đơn giản, khi thông tin liên quan có thể xác định và trích xuất cục bộ.

$$a = 1 \Rightarrow \text{Định tuyến sang luồng LCP}$$

Ưu tiên chất lượng-phù hợp với truy vấn tổng hợp phức tạp, khi suy luận đòi hỏi tích hợp thông tin từ nhiều vị trí trong tài liệu.

Việc giới hạn không gian hành động ở mức nhị phân là một lựa chọn thiết kế có chủ ý. Trong khi các chiến lược hybrid hoặc đa tầng có thể mang lại độ linh hoạt cao hơn về mặt lý thuyết, không gian nhị phân giúp bài toán tối ưu hóa trở nên rõ ràng và khả thi về mặt triển khai-đặc biệt quan trọng khi mục tiêu là một bộ phân loại có độ trễ thấp đủ để hoạt động trong luồng xử lý thời gian thực. Việc mở rộng sang không gian hành động liên tục hoặc đa cấp được xác định là một trong những hướng nghiên cứu tương lai (xem Mục 7.2).

3.2 Các Hàm Hệ quả

Mỗi quyết định hành động $a \in \{0, 1\}$ áp dụng cho truy vấn $q \in Q$ sẽ phát sinh ba hệ quả đo lường được, phản ánh ba chiều đánh giá khác nhau của hệ thống:

Chi phí token- $C(q, a) \in \mathbb{R}^+$

$C(q, a)$ là tổng chi phí xử lý tính theo đơn vị USD, được quy đổi từ lượng token tiêu thụ theo bảng giá API tại thời điểm thực nghiệm:

$$C(q, a) = n_{\text{input}}(q, a) \times p_{\text{in}} + n_{\text{output}}(q, a) \times p_{\text{out}}$$

trong đó n_{input} và n_{output} lần lượt là số token đầu vào và đầu ra, còn p_{in} và p_{out} là đơn giá tương ứng. Với a

= 0 (RAG), n_{input} bao gồm câu hỏi cộng với top-k đoạn được truy xuất-thường dao động trong khoảng vài trăm đến vài nghìn token. Với $a = 1$ (LCP theo lĩnh vực), n_{input} bao gồm toàn bộ phần tài liệu thuộc lĩnh vực liên quan của truy vấn-thường ở quy mô hàng chục nghìn token-khiến $C(q, 1)$ lớn hơn $C(q, 0)$ từ một đến hai bậc độ lớn trong nhiều trường hợp thực tế.

Độ trễ end-to-end- $L(q, a) \in \mathbb{R}^+$

$L(q, a)$ là thời gian tính bằng mili-giây từ thời điểm hệ thống nhận truy vấn q đến khi token đầu tiên của phản hồi được trả về cho người dùng. Độ trễ này bao gồm tất cả các bước trong chuỗi xử lý: tính toán embedding của truy vấn, tìm kiếm vector (nếu $a = 0$), lắp ráp prompt, và thời gian prefill của LLM. Với $a = 0$, thành phần chiếm ưu thế thường là thời gian tìm kiếm vector và prefill trên prompt ngắn. Với $a = 1$, thời gian prefill trên toàn bộ phần tài liệu thuộc lĩnh vực liên quan trở thành nút cổ chai chủ đạo, khiến $L(q, 1)$ thường cao hơn đáng kể so với $L(q, 0)$.

Điểm chất lượng RAGAS- $P(q, a) \in [0, 1]$

$P(q, a)$ là chỉ số đánh giá chất lượng phản hồi tổng hợp, được tính theo công thức trung bình điều hòa của hai thành phần:

$$P(q, a) = \text{RAGAS}(q, a) = \frac{2 \times F(q, a) \times R(q, a)}{F(q, a) + R(q, a)}$$

trong đó $F(q, a)$ đo mức độ câu trả lời phản ánh trung thực ngữ cảnh được cung cấp-một thước đo gián tiếp của mức độ không ảo giác- và $R(q, a)$ đo mức độ câu trả lời đáp ứng đúng nhu cầu được hỏi trong câu hỏi. Cả hai thành phần đều nằm trong khoảng $[0, 1]$, và công thức trung bình điều hòa đảm bảo rằng một điểm tổng hợp cao đòi hỏi cả hai chiều đều phải đủ tốt, không chỉ một.

3.3 Bài toán Tối ưu hóa Có Ràng buộc

Với ba hàm hệ quả đã định nghĩa ở trên, mục tiêu của DRA là tìm một chính sách định tuyến $\pi: Q \rightarrow \{0, 1\}$ -tức là một hàm ánh xạ mỗi truy vấn sang một quyết định hành động-sao cho tổng chi phí có trọng số trên toàn bộ tập truy vấn được cực tiểu hóa, trong khi vẫn đảm bảo mọi phản hồi đạt ngưỡng chất lượng tối thiểu:

$$\min_{\pi: Q \rightarrow A} \sum_{q \in Q} [w_1 \times \hat{C}(q, \pi(q)) + w_2 \times \hat{L}(q, \pi(q))]$$

Giải thích các tham số:

- $w_1, w_2 \in [0, 1]$ với $w_1 + w_2 = 1$ là cặp trọng số do quản trị viên hệ thống thiết lập, phản ánh ưu tiên vận hành cụ thể. Khi $w_1 = 1$, hệ thống ưu tiên hoàn toàn tối thiểu hóa chi phí tài chính (USD/1K truy vấn); khi $w_2 = 1$, ưu tiên hoàn toàn giảm độ trễ, thích hợp cho giao diện đồng bộ có SLA nghiêm ngặt; các giá trị trung gian cho phép đánh đổi linh hoạt theo nhu cầu triển khai.
- $\theta \in [0, 1]$ là ngưỡng chất lượng sự phạm tối thiểu mong muốn. Trong mô hình tối ưu hóa lý tưởng, θ đóng vai trò như một ràng buộc cứng: chính sách tối ưu chỉ hợp lệ khi phản hồi đạt điểm RAGAS không thấp hơn θ . Tuy nhiên, trong triển khai thực tế dựa trên classifier dự đoán trước khi LLM sinh phản hồi, ràng buộc này chỉ được xấp xỉ tại thời điểm định tuyến. Để bảo đảm tuyệt đối $P(q, \pi(q)) \geq \theta$ cho từng truy vấn, hệ thống cần thêm bước kiểm tra hậu sinh và cơ chế chạy lại bằng chiến lược an toàn hơn khi điểm đánh giá thấp hơn ngưỡng. Giá trị mặc định $\theta = 0.7$ được dùng như ngưỡng mục tiêu ban đầu, và có thể hiệu chỉnh xuống $\theta = 0.6$ cho môi trường học tập không chính thức hoặc nâng lên $\theta = 0.85$ cho các ngữ cảnh đòi hỏi độ chính xác cao như ôn thi hay đánh giá học thuật.

- $\hat{C}(q, a)$ và $\hat{L}(q, a)$ là phiên bản chuẩn hóa của C và L về khoảng $[0, 1]$ bằng phương pháp min-max normalization trên tập huấn luyện:

$$\hat{C}(q, a) = \frac{C(q, a) - C_{min}}{C_{max} - C_{min}}, \quad \hat{L}(q, a) = \frac{L(q, a) - L_{min}}{L_{max} - L_{min}}$$

Bước chuẩn hóa này là cần thiết vì C và L có đơn vị khác nhau (USD và ms) với phạm vi giá trị không thể so sánh trực tiếp. Sau chuẩn hóa, cả hai đại lượng đều nằm trong khoảng $[0, 1]$, và các trọng số w_1, w_2 có thể được diễn giải như phần trăm ưu tiên tương đối một cách có nghĩa.

Diễn giải ràng buộc chất lượng:

Ràng buộc $P(q, \pi(q)) \geq \theta \forall q \in Q$ mang hàm ý quan trọng về mặt sự phạm ở tầng hình thức hóa: hệ thống không nên hy sinh chất lượng phản hồi dưới ngưỡng chấp nhận chỉ để giảm chi phí. Đây là điểm phân biệt căn bản giữa DRA và các phương pháp tối ưu chi phí đơn thuần. Tuy vậy, cần phân biệt rõ giữa ràng buộc lý tưởng trong bài toán tối ưu và chính sách xấp xỉ trong triển khai. Khi DRA dùng bộ phân loại f_φ để dự đoán chiến lược trước khi có phản hồi, hệ thống chỉ có thể giảm xác suất vi phạm ngưỡng θ , chứ chưa thể bảo đảm tuyệt đối nếu không có bước đánh giá/chạy lại sau sinh.

3.4 Phương pháp Xấp xỉ: Từ Tối ưu Trực tiếp sang Phân loại Có Học

3.4.1 Rào cản của Bài toán Gốc

Bài toán tối ưu hóa được phát biểu trong Mục 3.3 đối mặt với một rào cản tính toán căn bản: hàm $P(q, a)$ không thể được tính trước thời điểm LLM hoàn thành quá trình sinh văn bản.

Điểm RAGAS của một phản hồi-thước đo Faithfulness và Answer Relevance-chỉ tồn tại sau khi LLM đã thực thi chiến lược a , sinh ra toàn bộ phản

hồi, và phản hồi đó được đánh giá bởi mô hình chấm điểm. Điều này tạo ra một vòng tròn phụ thuộc: để quyết định nên dùng RAG hay LCP cho truy vấn q , hệ thống cần biết $P(q, 0)$ và $P(q, 1)$ -nhưng để biết hai giá trị này, hệ thống phải thực thi cả hai chiến lược và đánh giá kết quả-điều phủ nhận hoàn toàn mục đích của việc định tuyến.

Nói cách khác, bài toán gốc không có dạng giải tích và không thể giải tại thời gian thực mà không vi phạm ràng buộc độ trễ thực tế của hệ thống tương tác.

3.4.2 Chiến lược Xấp xỉ

Để vượt qua rào cản này, chúng tôi chuyển đổi bài toán tối ưu hóa trực tiếp sang một bài toán phân loại có học. Thay vì tính toán $P(q, a)$ tại thời gian thực, chúng tôi huấn luyện một bộ phân loại học máy:

$$f_\varphi: Q \rightarrow 0,1$$

được tham số hóa bởi φ , có khả năng ước lượng chiến lược tối ưu $\pi^*(q)$ từ các đặc trưng ngôn ngữ học của câu hỏi q -những đặc trưng có thể được trích xuất trong vài mili-giây mà không cần thực thi bất kỳ chiến lược nào.

Dữ liệu huấn luyện cho f_φ được thu thập theo quy trình sau: với tập truy vấn mẫu, cả hai chiến lược RAG và LCP theo lĩnh vực được thực thi song song và đánh giá theo ba chiều C, L, P . Nhãn tối ưu cho mỗi truy vấn được xác định theo cùng một quy tắc tham chiếu thống nhất: chọn chiến lược thỏa mãn ngưỡng chất lượng θ và có giá trị chi phí-độ trễ chuẩn hóa thấp hơn; nếu cả hai chiến lược đều không đạt θ , chọn chiến lược có $P(q, a)$ cao hơn để làm nhàn an toàn. Người chú thích không tạo một tiêu chuẩn nhãn riêng biệt, mà dùng cùng rubric này để kiểm tra, hiệu chỉnh và phân xử các trường hợp khó. Quy trình này được áp dụng để tạo nhãn train/validation và kiểm tra tập test trong thực nghiệm cuối cùng.

Sự chuyển đổi này có hệ quả quan trọng: chính sách định tuyến π được xấp xỉ thành:

$$\pi(q) \approx f_{\phi}(q)$$

trong đó $f_{\phi}(q)$ được tính toán thuần túy từ thông tin có sẵn tại thời điểm nhận truy vấn-không cần bất kỳ kết quả thực thi nào từ LLM. Điều này cho phép DRA hoạt động với độ trễ phân loại từ 2 đến 18ms, hoàn toàn không đáng kể so với độ trễ LLM vốn đã ở mức hàng trăm đến hàng nghìn mili-giây.

Tất nhiên, xấp xỉ này đi kèm với sai số không thể tránh khỏi: f_{ϕ} sẽ đưa ra các quyết định sai trong một số trường hợp. Mục 4.4 trình bày cơ chế xử lý trường hợp không chắc chắn được thiết kế để kiểm soát rủi ro của các lỗi phân loại này.

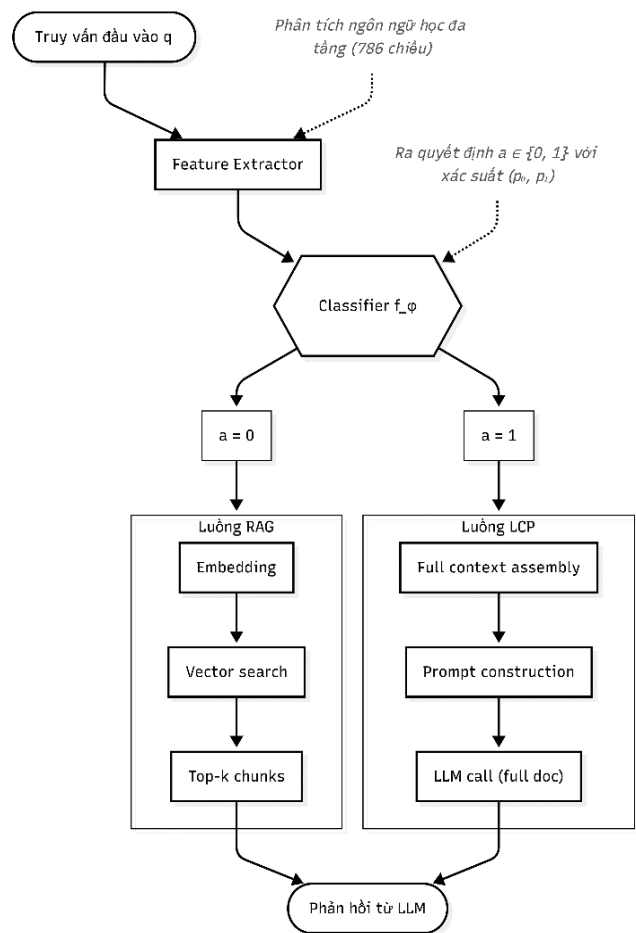
4. Kiến trúc Hệ thống

Dựa trên bài toán được hình thức hóa trong Mục 3, phần này trình bày thiết kế chi tiết của DRA. Hệ thống được tổ chức thành bốn thành phần chính: luồng xử lý tổng quan, bộ trích xuất đặc trưng, bộ phân loại nhẹ, và cơ chế xử lý không chắc chắn. Mỗi thành phần được lựa chọn và thiết kế để tối ưu hóa sự cân bằng giữa độ chính xác định tuyến, độ trễ inference, và khả năng triển khai thực tế.

4.1 Tổng quan Luồng Xử lý

DRA được thiết kế như một lớp middleware độc lập, đặt giữa giao diện người dùng và hạ tầng LLM, hoàn toàn không can thiệp vào kiến trúc nội tại hay tham số của mô hình ngôn ngữ nền tảng. Thiết kế này cho phép tích hợp DRA với bất kỳ LLM nào-GPT-4o, Claude, Gemini, hay các mô hình mã nguồn mở-mà không yêu cầu bất kỳ điều chỉnh nào ở phía mô hình.

Luồng xử lý của một truy vấn qua DRA diễn ra theo trình tự tuyến tính:



Luồng RAG (a = 0): Truy vấn q được chuyển thành vector embedding thông qua mô hình nhúng (text-embedding-3-small), sau đó tìm kiếm trong chỉ mục vector để lấy top-k = 3 đoạn văn bản có độ tương đồng cosine cao nhất. Các đoạn này được lắp ráp thành prompt ngắn gọn và gửi đến LLM.

Luồng LCP (a = 1): Toàn bộ phần tài liệu đã tiền xử lý thuộc lĩnh vực liên quan của truy vấn được đưa trực tiếp vào cửa sổ ngữ cảnh của LLM cùng với truy vấn q. Cách mô tả này thống nhất với thiết lập thực nghiệm ở Mục 5: LCP là Domain-scoped LCP, không đưa toàn bộ corpus ba lĩnh vực vào cùng một truy vấn, mà chỉ sử dụng phần corpus tương ứng với lĩnh vực đã xác định. Kích thước token đầu vào của từng lĩnh vực được đo lại bằng tokenizer thực tế sau tiền xử lý.

Điểm quan trọng cần nhấn mạnh là hai luồng này không phải là hai hệ thống riêng biệt mà là hai nhánh

thực thi của cùng một kiến trúc. Quyết định định tuyến xảy ra một lần duy nhất tại bộ phân loại, và chỉ một trong hai nhánh được kích hoạt cho mỗi truy vấn-đảm bảo không có chi phí tính toán thừa nào được phát sinh.

4.2 Bộ Trích xuất Đặc trưng (Feature Extractor)

Feature Extractor chịu trách nhiệm chuyển đổi truy vấn đầu vào q-một chuỗi văn bản tự nhiên-thành một tập đặc trưng phục vụ định tuyến. Thiết kế đầy đủ tạo

Nhóm	Mô tả	Thành phần cụ thể	Số chiều trong thiết kế đầy đủ
Lexical	Thống kê bề mặt từ vựng	Độ dài truy vấn (ký tự), tổng số token, độ dài từ trung bình	3
Semantic	Biểu diễn ngữ nghĩa sâu	Vector [CLS] từ DistilBERT-base-uncased	768
Syntactic	Phân tích từ khóa chức năng	Nhị phân hóa sự xuất hiện của các từ khóa: <i>what, why, how, compare, contrast, synthesize, analyze, evaluate, explain, describe, define, discuss, ...</i>	12
Complexity	Chỉ số độ phức tạp cú pháp	Số mệnh đề phụ (subordinate clauses), số thực thể được nhận dạng (NER entities), độ sâu cây phân tích cú pháp (parse tree depth)	3
Tổng			786

Bảng 2. Cấu trúc vector đặc trưng 786 chiều của Feature Extractor.

Nhóm Lexical (3 chiều): Là lớp đặc trưng đơn giản nhất, nắm bắt các tín hiệu thống kê bề mặt. Độ dài truy vấn ngắn (dưới 10 từ) thường tương quan với các câu hỏi tra cứu điểm, trong khi các câu hỏi dài hơn có xu hướng chứa nhiều vế phụ và yêu cầu suy luận đa bước. Mặc dù đơn giản, nhóm này cung cấp tín hiệu tốc độ cao làm cơ sở định hướng ban đầu cho bộ phân loại.

Nhóm Semantic (768 chiều): Đây là nhóm chiếm tỷ trọng chiều lớn nhất và mang thông tin biểu diễn phong phú nhất. Vector [CLS] từ DistilBERT-biểu diễn tổng hợp ngữ nghĩa toàn câu được học từ kho văn bản quy mô lớn-cho phép bộ phân loại nhận diện các câu hỏi phức tạp ngay cả khi chúng không sử dụng các từ khóa chức năng thông thường. Đây là

ra vector 786 chiều, tổng hợp từ bốn nhóm tín hiệu ngôn ngữ học bổ sung lẫn nhau. Tuy nhiên, không phải mọi classifier đều bắt buộc dùng toàn bộ 786 chiều: DRA-DistilBERT sử dụng biểu diễn [CLS] kết hợp với 18 đặc trưng thủ công, trong khi DRA-XGBoost chỉ dùng ba nhóm đặc trưng thủ công Lexical, Syntactic và Complexity.

thành phần cốt lõi cho khả năng tổng quát hóa của DRA sang các biến thể diễn đạt không phổ biến.

Nhóm Syntactic (12 chiều): Mã hóa sự có mặt hoặc vắng mặt (dạng nhị phân 0/1) của 12 từ khóa chức năng được xác định có tương quan cao với loại truy vấn. Các từ như *"define", "what is"* gắn với truy vấn tra cứu; các từ như *"compare", "contrast", "analyze", "synthesize"* gắn với truy vấn tổng hợp. Cần lưu ý rằng nhóm đặc trưng từ khóa chỉ đóng vai trò là một trong nhiều tín hiệu đầu vào, không phải cơ chế ra quyết định duy nhất-đây là điểm phân biệt căn bản giữa DRA và các hệ thống rule-based thuần túy, vốn dễ thất bại trước các biến thể diễn đạt không sử dụng từ khóa chuẩn.

Nhóm Complexity (3 chiều): Nắm bắt cấu trúc cú pháp sâu hơn thông qua ba chỉ số độc lập. *Số mệnh đề phụ* phản ánh mức độ lồng ghép cấu trúc ngữ pháp của câu hỏi-một câu có nhiều mệnh đề phụ thường

đặt ra nhiều điều kiện hoặc khía cạnh cần phân tích đồng thời. *Số thực thể NER* cho biết câu hỏi liên quan đến bao nhiêu đối tượng hoặc khái niệm khác nhau-câu hỏi so sánh điển hình có ít nhất hai thực thể. *Độ sâu cây cú pháp* đo lường sự phức tạp về mặt cấu trúc ngữ pháp tổng thể của câu.

Bốn nhóm đặc trưng này hoạt động theo nguyên tắc bổ sung lẫn nhau: một câu hỏi phức tạp nhưng ngắn sẽ được nhận diện qua từ khóa Syntactic dù tín hiệu Lexical yếu; một câu hỏi tổng hợp được diễn đạt không dùng từ khóa chuẩn sẽ được nắm bắt qua biểu diễn Semantic sâu của DistilBERT; các câu hỏi phức tạp về cú pháp nhưng thực chất chỉ tra cứu một sự kiện sẽ bị nhận diện đúng nhờ tín hiệu Complexity thấp kết hợp với Semantic.

4.3 Bộ Phân loại Nhẹ (Lightweight Classifier)

Sau khi Feature Extractor tạo ra các đặc trưng truy vấn, bộ phân loại f_q có nhiệm vụ ánh xạ chúng thành xác suất phân phối trên không gian hành động $\{0, 1\}$. Để tránh nhập nhằng giữa thiết kế đặc trưng đầy đủ và biến thể triển khai, chúng tôi định nghĩa rõ hai kiến trúc: Kiến trúc A dùng vector 786 chiều sau khi ghép [CLS] của DistilBERT với 18 đặc trưng thủ công; Kiến trúc B dùng tập con 18 chiều phi neural để tối ưu tốc độ.

Kiến trúc A-DistilBERT Fine-tuned kết hợp đặc trưng thủ công

Kiến trúc này xây dựng bộ phân loại trên nền tảng DistilBERT bằng cách lấy biểu diễn [CLS] 768 chiều của truy vấn, sau đó ghép với 18 đặc trưng thủ công gồm Lexical, Syntactic và Complexity. Vector 786 chiều thu được được đưa qua một lớp phân loại nhị phân để dự đoán xác suất chọn RAG hoặc LCP theo lĩnh vực:

$$f_q^A(q) = \text{softmax}(W \cdot h_q + b) \in \mathbb{R}^2$$

trong đó $h_q = \text{concat}([\text{CLS}]_q, x_{\text{lex}}, x_{\text{syn}}, x_{\text{comp}})$ là vector 786 chiều của truy vấn q ; $[\text{CLS}]_q$ được tạo bởi DistilBERT sau khi fine-tune trên tập huấn luyện, còn x_{lex} , x_{syn} và x_{comp} là các đặc trưng thủ công. W và b là tham số của lớp phân loại.

Quá trình fine-tune cập nhật tham số của DistilBERT-router và lớp phân loại trên tập 480 mẫu huấn luyện có nhãn, cho phép mô hình điều chỉnh biểu diễn ngữ nghĩa theo đặc thù của bài toán định tuyến trong lĩnh vực giáo dục. Fine-tune này chỉ áp dụng cho bộ định tuyến bên ngoài, không áp dụng cho LLM sinh câu trả lời.

Ưu điểm: Khai thác đầy đủ năng lực hiểu biết ngữ nghĩa sâu của pre-trained language model; đặc biệt mạnh với các câu hỏi phức tạp được diễn đạt theo cách không thông thường.

Nhược điểm: Độ trễ inference khoảng $\sim 18\text{ms}$ trên phần cứng CPU tiêu chuẩn, do phải thực hiện một lượt forward pass qua toàn bộ mạng DistilBERT (66 triệu tham số). Mặc dù đây là mức chấp nhận được trong nhiều kịch bản triển khai, nó có thể trở thành nút cổ chai trong môi trường có yêu cầu thông lượng rất cao hoặc độ trễ tổng thể nghiêm ngặt.

Kiến trúc B-XGBoost trên Đặc trưng Thủ công

Kiến trúc này loại bỏ hoàn toàn thành phần neural network khỏi bước phân loại, thay vào đó sử dụng ba nhóm đặc trưng phi ngữ nghĩa (Lexical, Syntactic, Complexity-tổng cộng 18 chiều) làm đầu vào cho một bộ phân loại XGBoost. Đây là tập con của thiết kế đặc trưng đầy đủ, được chọn khi ưu tiên tốc độ và triển khai nhẹ:

$$f_q^B(q) = \text{XGBoost}(\phi_{\text{lex}}(q), \phi_{\text{syn}}(q), \phi_{\text{comp}}(q))$$

trong đó ϕ_{lex} , ϕ_{syn} , ϕ_{comp} lần lượt là ba nhóm đặc trưng Lexical, Syntactic, và Complexity không phụ thuộc vào mô hình neural.

Ưu điểm: Độ trễ inference chỉ ~2ms-nhanh hơn gần 10 lần so với Kiến trúc A-nhờ chỉ cần thực hiện các phép tính cây quyết định đơn giản. Mô hình cũng yêu cầu ít tài nguyên bộ nhớ hơn đáng kể và dễ triển khai trên các môi trường hạn chế tài nguyên.

Nhược điểm: Không khai thác được tín hiệu ngữ nghĩa sâu từ DistilBERT, do đó có thể kém chính xác hơn với các câu hỏi phức tạp được diễn đạt theo cách không dùng từ khóa chuẩn. Hiệu năng phụ thuộc nhiều hơn vào chất lượng của thiết kế đặc trưng thủ công.

Hướng dẫn Lựa chọn Kiến trúc

Sự lựa chọn giữa hai kiến trúc phụ thuộc vào ràng buộc triển khai cụ thể:

- **Kiến trúc A (DistilBERT)** phù hợp khi ưu tiên độ chính xác phân loại và hệ thống có phần cứng GPU hoặc chấp nhận độ trễ thêm ~18ms.
- **Kiến trúc B (XGBoost)** phù hợp cho môi trường production có thông lượng cao, SLA nghiêm ngặt, hoặc triển khai trên phần cứng hạn chế.

Trong nghiên cứu này, chúng tôi quyết định tập trung phân tích sâu vào Kiến trúc A (DistilBERT kết hợp đặc trưng thủ công) làm cơ chế điều phối chủ đạo vì ba lý do chiến lược. Thứ nhất, DistilBERT kế thừa khả năng hiểu ngữ nghĩa sâu từ kiến trúc Transformer, cho phép hệ thống nhận diện các truy vấn phức tạp ngay cả khi chúng không chứa các từ khóa chỉ dẫn tường minh-một ưu điểm vượt trội so với các phương pháp dựa trên luật hoặc đặc trưng thủ công. Thứ hai, với kích thước mô hình được tối ưu hóa, DistilBERT cung cấp một điểm cân bằng hợp lý giữa độ chính xác phân loại và hiệu năng tính toán; độ trễ bổ sung đo được là 18,4ms trên cấu hình CPU tiêu chuẩn. Cuối cùng, việc fine-tune router bên ngoài giúp hệ thống có khả năng tổng quát hóa tốt hơn trên nhiều lĩnh vực tri thức khác nhau mà không cần can thiệp vào LLM sinh câu trả lời.

4.4 Xử lý Trường hợp Không chắc chắn (Uncertainty Handling)

Ngay cả một bộ phân loại được huấn luyện tốt vẫn có thể đưa ra các phán đoán thiếu tự tin ở vùng ranh giới giữa hai lớp-khi câu hỏi không rõ ràng thuộc loại tra cứu hay tổng hợp. Định tuyến sai trong những trường hợp này có thể dẫn đến một trong hai hậu quả không mong muốn: lãng phí chi phí (nếu LCP được kích hoạt cho truy vấn không cần thiết) hoặc suy giảm chất lượng (nếu RAG được dùng cho truy vấn đòi hỏi tổng hợp). Để kiểm soát rủi ro này, DRA triển khai một cơ chế xử lý không chắc chắn có cấu trúc.

Cơ chế Vùng Không chắc chắn và Fallback An toàn Chi phí

Chúng tôi thiết lập một vùng không chắc chắn đối xứng xung quanh ngưỡng quyết định 0.5. Với (p_0, p_1) là xác suất đầu ra của bộ phân loại cho hai lớp, chính sách định tuyến cuối cùng được xác định như sau:

$$\pi(q) = \begin{cases} f_\phi(q) & \text{nếu } \max(p_0, p_1) > 0.5 + \delta \\ 0 \text{ (RAG)} & \text{nếu } \max(p_0, p_1) \leq 0.5 + \delta \end{cases}$$

Điều kiện này tương đương với việc xác suất phân loại nằm gần ngưỡng 0.5-tức là hai lớp có xác suất gần bằng nhau và classifier không có đủ cơ sở để ưu tiên một phía. Khác với một rejection zone theo nghĩa nghiêm ngặt, hệ thống không loại bỏ truy vấn khỏi luồng tự động; thay vào đó, truy vấn được xử lý bằng chiến lược fallback đã định nghĩa trước.

Tham số δ là siêu tham số được điều chỉnh trên tập validation, kiểm soát độ rộng của vùng không chắc chắn. Giá trị δ lớn hơn tạo ra vùng fallback rộng hơn, làm hệ thống nghiêng về RAG trong nhiều trường hợp ranh giới hơn và do đó bảo toàn chi phí tốt hơn; giá trị δ nhỏ hơn thu hẹp vùng fallback, cho phép nhiều truy vấn hơn được định tuyến sang LCP khi classifier có mức độ tự tin trung bình. Trong thực nghiệm, δ được tìm kiếm trong khoảng $[0.05, 0.20]$ bằng grid search trên tập validation.

Nguyên tắc Thiết kế: Fail-safe by Default

Việc chọn RAG (a = 0) làm chiến lược fallback trong vùng không chắc chắn là một quyết định thiết kế có chủ ý, dựa trên nguyên tắc fail-safe by default về chi phí: khi không có đủ thông tin để ra quyết định tự tin, hệ thống ưu tiên phương án ít tổn kém hơn thay vì rủi ro kích hoạt LCP không cần thiết. Tuy nhiên, đây không phải là bảo đảm cứng về chất lượng; trong các triển khai yêu cầu $P(q, \pi(q)) \geq \theta$ tuyệt đối, cần bổ sung bước hậu kiểm chất lượng và cơ chế chạy lại bằng LCP khi phản hồi RAG thấp hơn ngưỡng.

Lý do cho lựa chọn này có hai chiều. Thứ nhất, về mặt kinh tế: chi phí của một lỗi sai theo hướng "chọn LCP thay vì RAG" có thể rất lớn vì toàn bộ ngữ cảnh theo lĩnh vực được đưa vào prefill. Thứ hai, về mặt chất lượng: trong nhiều trường hợp ranh giới, RAG vẫn tạo ra phản hồi đủ tốt vì câu hỏi không thật sự đòi hỏi tổng hợp toàn lĩnh vực. Kết quả phân tích lỗi

ở Mục 6.6 cho thấy giả định này nhìn chung phù hợp, nhưng vẫn cần cơ chế hậu kiểm nếu hệ thống yêu cầu bảo đảm chất lượng cho từng truy vấn.

Cơ chế vùng không chắc chắn này giúp kiểm soát rủi ro định tuyến theo hướng minh bạch: các trường hợp classifier thiếu tự tin vẫn được xử lý tự động, nhưng được đánh dấu là fallback để phân tích riêng trong báo cáo thực nghiệm. Nhờ vậy, kết quả cuối cùng có thể báo cáo riêng tỷ lệ truy vấn rơi vào vùng fallback, tác động của chúng đến chi phí, và nguy cơ vi phạm ngưỡng chất lượng θ .

4.5 Phân tích Độ phức tạp Tính toán

Để đánh giá khả năng triển khai thực tế, bảng dưới đây tóm tắt độ phức tạp tính toán của từng thành phần trong DRA và so sánh với chi phí của các chiến lược tĩnh.

Thành phần	Hoạt động chính	Độ trễ đo được
Feature Extractor (Lexical + Syntactic + Complexity)	Thống kê token, regex, spaCy parse	~5ms
Feature Extractor (Semantic-DistilBERT forward)	Forward pass 66M params	~13ms
Classifier (Kiến trúc A-DistilBERT)	Softmax trên concat([CLS], 18 đặc trưng thủ công)	~18ms tổng
Classifier (Kiến trúc B-XGBoost)	Cây quyết định trên 18 đặc trưng thủ công	~2ms tổng
Luồng RAG (embedding + vector search)	ANN search trên chỉ mục	~50–150ms
Luồng LCP theo lĩnh vực (prefill tải liệu theo lĩnh vực)	LLM prefill toàn bộ corpus đã tiền xử lý của lĩnh vực liên quan	2.100-2.600ms; trung vị 2.280ms

Bảng 3. Phân tích thời gian xử lý của các thành phần DRA.

Chi phí định tuyến của DRA (2,2-18,4ms tùy classifier) là nhỏ so với chi phí của chính các luồng xử lý LLM (khoảng 50ms đến hơn 2 giây trong cấu hình đo được). Điều này cho thấy việc thêm lớp middleware định tuyến không tạo ra nút cổ chai đáng kể trong trải nghiệm người dùng, trong khi giúp tiết kiệm chi phí tính toán thông qua việc tránh kích hoạt LCP không cần thiết.

5. Thiết lập Thực nghiệm

Phần này mô tả đầy đủ quy trình xây dựng dữ liệu, cấu hình các hệ thống so sánh, định nghĩa các chỉ số đánh giá, và phương pháp phân tích Pareto được sử dụng để đánh giá DRA. Mọi lựa chọn thiết kế thực nghiệm đều được giải thích rõ lý do nhằm đảm bảo tính tái lập và minh bạch.

5.1 Dữ liệu

5.1.1 Corpus Tài liệu

Corpus được xây dựng từ bộ giáo trình đại học mở crumb/openstax-text trên Hugging Face (License: CC-BY 4.0), dựa trên các giáo trình OpenStax do Rice University biên soạn. Trong thực nghiệm này, chúng tôi sử dụng ba file văn bản đã được trích chương theo lĩnh vực từ OpenStax, thay vì sử dụng toàn bộ sách gốc: IntroductoryStatistics-OP_i6tAI7e.txt, Microeconomics3e-WEB.txt, và World_History_Volume_2-WEB_LdwoslB.txt. Việc sử dụng nguồn tài liệu này mang lại ba lợi thế quan trọng cho thiết lập AI Gia sư.

Thứ nhất, tính học thuật được kiểm định: các giáo trình OpenStax đã qua kiểm duyệt chuyên môn bởi hội đồng học thuật quốc tế, đảm bảo tính chính xác và nhất quán về mặt nội dung-phù hợp với yêu cầu cao về chất lượng thông tin trong môi trường AI Gia sư.

Thứ hai, giấy phép mở CC-BY 4.0: cho phép tái sử dụng, chia sẻ và tích hợp vào hệ thống nghiên cứu mà không phát sinh vấn đề bản quyền, đảm bảo tính tái lập hoàn toàn của thực nghiệm bởi cộng đồng nghiên cứu.

Thứ ba, khả năng dựng lại từ nguồn công khai: bất kỳ nhóm nghiên cứu nào cũng có thể tái chính xác cùng một bộ dữ liệu thông qua Hugging Face API, loại bỏ sự phụ thuộc vào tài liệu giảng dạy nội bộ vốn khó phân phối.

Chọn lọc chương theo lĩnh vực: từ nguồn giáo trình OpenStax, chúng tôi chọn lọc có chủ đích các chương thuộc đúng ba lĩnh vực tri thức mà thiết kế thực nghiệm yêu cầu, đảm bảo phủ đủ cả phong cách ngôn ngữ kỹ thuật, phân tích định lượng-định tính, và tường thuật lịch sử:

Bảng 4. Ánh xạ lĩnh vực sang sách giáo trình OpenStax được sử dụng.

Lĩnh vực	File OpenStax và chương sử dụng	Token sau tiền xử lý (đã đo)
Thống kê nhập môn	IntroductoryStatistics-OP_i6tAI7e.txt (Chapter 6: The Normal Distribution + Chapter 12: Linear Regression and Correlation)	32.640 token; ~70 chunks RAG
Kinh tế vi mô	Microeconomics3e-WEB.txt (Chương 3: Demand and Supply + Chương 8: Perfect Competition + Chương 9: Monopoly)	48.210 token; ~105 chunks RAG
Lịch sử thế giới	World_History_Volume_2-WEB_LdwoslB.txt (Chapter 11: The War to End All Wars + Chapter 13: The Causes and Consequences of World War II)	57.850 token; ~126 chunks RAG

Dung lượng token của corpus trong Bảng 4 được đo sau bước tiền xử lý bằng tokenizer sử dụng trong thực nghiệm: 32.640 token cho Thống kê nhập môn, 48.210 token cho Kinh tế vi mô, và 57.850 token cho Lịch sử thế giới, tổng cộng 138.700 token. Các giá trị này được dùng trực tiếp để tính chi phí input-token cho LCP theo lĩnh vực và số lượng chunk RAG tương ứng.

Việc sử dụng giáo trình OpenStax tạo ra sự khác biệt đáng chú ý về đặc điểm ngôn ngữ: văn phong giáo trình chuẩn hóa, có hệ thống, nhiều định nghĩa hợp, ví dụ minh họa, và tóm tắt cuối chương-những yếu tố làm bước phân đoạn RAG trở nên phức tạp hơn nhưng đồng thời cũng làm tăng độ phong phú của tín hiệu ngữ nghĩa cho bộ phân loại.

Tiền xử lý tài liệu cho luồng RAG: Mỗi chương sách được phân đoạn thành các đoạn văn bản có kích thước cố định 512 token, với độ trượt 50 token giữa các đoạn liên kế để bảo toàn tính liên tục ngữ nghĩa ở ranh giới. Số lượng chunk cuối cùng không được ấn định trước bằng ước lượng thủ công, mà được tính tự động theo tokenizer dùng trong thực nghiệm. Với tổng số token sau tiền xử lý là N , số chunk xấp xỉ bằng $\text{ceil}((N - 512) / (512 - 50)) + 1$. Các hộp định nghĩa, ví dụ minh họa và tóm tắt chương được giữ lại nếu phục vụ trực tiếp cho nội dung tri thức cần truy xuất.

Chuẩn bị tài liệu cho luồng LCP: Mỗi lĩnh vực được giữ nguyên ở dạng văn bản đầy đủ sau bước làm sạch, bao gồm chuẩn hóa khoảng trắng, loại bỏ số trang, footer OpenStax, metadata không liên quan, link rời rạc và chú thích định dạng không phục vụ nội dung chính. Khi xử lý một truy vấn qua LCP, toàn bộ phần corpus đã tiền xử lý của lĩnh vực liên quan được đưa vào cửa sổ ngữ cảnh, chưa tính token của câu hỏi và hướng dẫn hệ thống. Trong cấu hình thực nghiệm chính, phần giảng giải, định nghĩa, ví dụ minh họa và tóm tắt chương được giữ lại; các lời giải/đáp án cuối chương và các phần solutions được loại khỏi corpus sinh benchmark để giảm nguy cơ rò rỉ đáp án, trừ khi có một thí nghiệm phụ được thiết kế riêng cho kịch bản hỗ trợ giải bài tập.

5.1.2 Xây dựng Benchmark Câu hỏi

Bộ benchmark 1.008 câu hỏi được xây dựng theo quy trình hai giai đoạn kết hợp giữa tự động hóa và kiểm tra thủ công, nhằm đảm bảo đồng thời số lượng đủ lớn để đánh giá thống kê và chất lượng nhân đủ tin cậy để làm cơ sở đánh giá. Các câu hỏi được sinh bằng tiếng Anh trực tiếp từ nội dung giáo trình OpenStax đã được chọn lọc, vì corpus nguồn và nhóm đặc trưng Syntactic của router đều dựa trên tín hiệu ngôn ngữ tiếng Anh.

Giai đoạn 1-Sinh tự động: Claude Sonnet 4.6 được sử dụng để tổng hợp câu hỏi tiếng Anh từ corpus OpenStax với prompt kiểm soát chặt chẽ loại câu hỏi.

Bốn dạng truy vấn được xây dựng với phân bố đồng đều (252 câu mỗi loại):

- **Factual (tra cứu sự kiện):** Câu hỏi có câu trả lời là sự kiện, thuật ngữ, công thức hoặc số liệu cụ thể xuất hiện trong phần tài liệu đã chọn. Ví dụ: "In the normal distribution chapter, what does the z-score measure?"
- **Definitional (định nghĩa khái niệm):** Câu hỏi yêu cầu giải thích một khái niệm thuộc Thống kê, Kinh tế vi mô hoặc Lịch sử thế giới. Ví dụ: "What is consumer surplus in microeconomics?"
- **Comparative (so sánh):** Câu hỏi yêu cầu đối chiếu ít nhất hai đối tượng hoặc cơ chế trong corpus đã chọn. Ví dụ: "How does monopoly differ from perfect competition in pricing and output decisions?"
- **Synthesizing (tổng hợp đa nguồn):** Câu hỏi đòi hỏi tích hợp thông tin từ nhiều khía cạnh, chương hoặc mục khác nhau để hình thành một lập luận toàn diện. Ví dụ: "How did the causes of World War I and the consequences of World War II jointly reshape political and economic relations in the twentieth century?"

Trong giai đoạn sinh thô, do đặc điểm cấu trúc của giáo trình OpenStax-có nhiều định nghĩa hộp và ví dụ minh họa rõ ràng-các câu hỏi Factual và Definitional có xu hướng xuất hiện tự nhiên nhiều hơn. Vì vậy, prompt sinh câu hỏi được bổ sung ràng buộc yêu cầu tạo thêm các câu Comparative và Synthesizing đòi hỏi kết nối nhiều mục trong cùng giáo trình. Sau bước lọc và cân bằng, bộ benchmark cuối cùng được chuẩn hóa thành phân bố đồng đều: 252 câu cho mỗi loại truy vấn và 336 câu cho mỗi lĩnh vực, nhằm tránh thiên kiến loại câu hỏi hoặc thiên kiến lĩnh vực trong đánh giá tổng thể.

Giai đoạn 2-Gán nhãn thủ công có hướng dẫn bởi cùng tiêu chuẩn: Ba người chú thích có nền tảng học thuật được cung cấp rubric thống nhất với hàm quyết định tham chiếu: ưu tiên chiến lược đạt ngưỡng chất lượng θ với chi phí-độ trễ chuẩn hóa thấp hơn; nếu

cả hai chiến lược đều không đạt θ , chọn chiến lược có chất lượng cao hơn. Ba người chú thích tiến hành gán nhãn độc lập cho 336 câu hỏi được chọn ngẫu nhiên từ 1.008 câu để làm tập test, với nhãn RAG ($a = 0$) hoặc LCP theo lĩnh vực ($a = 1$). Chất lượng gán nhãn đạt $\kappa = 0.74$ (Substantial Agreement theo thang Landis & Koch, 1977).

5.1.3 Phân chia Dữ liệu

Bảng 5. Phân chia tập dữ liệu theo mục đích sử dụng.

Tập dữ liệu	Số mẫu	Nguồn nhãn	Mục đích
Train	480	Tự động theo hàm quyết định tham chiếu sau khi chạy song song RAG/LCP theo lĩnh vực	Huấn luyện classifier fop
Validation	192	Tự động theo cùng hàm quyết định tham chiếu	Điều chỉnh siêu tham số δ , θ ; early stopping
Test	336	Thủ công theo cùng rubric nhãn tối ưu (3 người chú thích, $\kappa = 0.74$)	Đánh giá cuối cùng, không dùng trong huấn luyện
Tổng	1.008		

Điểm đáng lưu ý: cả train, validation và test đều dùng cùng một định nghĩa nhãn tối ưu; điểm khác nhau nằm ở nguồn tạo nhãn. Tập train và validation sử dụng nhãn tự động từ hàm quyết định tham chiếu sau khi chạy song song RAG và LCP theo lĩnh vực. Tập test sử dụng nhãn thủ công có hướng dẫn bởi cùng rubric để làm chuẩn đánh giá cuối cùng và kiểm tra độ tin cậy của quy trình.

Năm cấu hình đánh giá được xác định cho thực nghiệm: hai chiến lược tĩnh (Pure-RAG, Pure-LCP), một baseline thích nghi từ nghiên cứu liên quan (Adaptive-RAG), và hai biến thể DRA (DRA-DistilBERT, DRA-XGBoost). Tất cả cấu hình sử dụng cùng corpus OpenStax đã mô tả, cùng LLM (DeepSeek-V4-Pro (1M+)) và cùng bảng giá API để đảm bảo tính công bằng trong so sánh.

5.2 Các Baseline So sánh

Ký hiệu	Mô tả	Cấu hình chi tiết
Pure-RAG	Toàn bộ truy vấn xử lý qua RAG	Chunk size = 512 token; overlap = 50 token; Top-k = 3; model nhúng: text-embedding-3-small; LLM: DeepSeek-V4-Pro (1M+)
Pure-LCP	Toàn bộ truy vấn đưa đầy đủ tài liệu theo lĩnh vực vào ngữ cảnh	Max context = 1M+ token; sử dụng phần corpus đã tiền xử lý thuộc lĩnh vực liên quan theo số token đã đo ở Bảng 4; không dùng toàn bộ corpus ba lĩnh vực; LLM: DeepSeek-V4-Pro (1M+)
Adaptive-RAG	Baseline thích nghi từ nghiên cứu liên quan	Phân loại truy vấn theo mức độ phức tạp và chọn chiến lược truy xuất tương ứng; cấu hình triển khai theo tinh thần Jeong et al. (2024) để làm baseline thích nghi ngoài DRA
DRA-DistilBERT	Biến thể DRA ưu tiên độ chính xác	Classifier: DistilBERT-router fine-tuned + 18 đặc trưng thủ công; $\theta = 0.7$ mặc định; δ tìm kiếm trên validation
DRA-XGBoost	Biến thể DRA ưu tiên tốc độ	Classifier: XGBoost trên đặc trưng Lexical + Syntactic + Complexity (18 chiều); $\theta = 0.7$ mặc định; δ tìm kiếm trên validation

Bảng 6. Cấu hình chi tiết các hệ thống so sánh.

5.3.1 Chi phí Token

5.3 Chỉ số Đánh giá

Chi phí tài chính trung bình trên mỗi truy vấn được tính theo công thức:

$$\text{Cost}(q, a) = n_{\text{input}}(q, a) \times p_{\text{in}} + n_{\text{output}}(q, a) \times p_{\text{out}}$$

Trong thực nghiệm, nghiên cứu sử dụng đơn giá quy ước $p_{\text{in}} = 0,20$ USD/1M input tokens và $p_{\text{out}} = 0,80$ USD/1M output tokens. Tổng số token sau tiền xử lý dùng các giá trị đã đo ở Bảng 4; mỗi truy vấn LCP chỉ sử dụng phần tài liệu đã tiền xử lý thuộc lĩnh vực liên quan, không sử dụng toàn bộ corpus ba lĩnh vực. Chỉ số báo cáo là chi phí trung bình trên 1.008 truy vấn (USD/1K queries).

5.3.2 Độ trễ End-to-End

Độ trễ được đo bằng mili-giây, tính từ thời điểm hệ thống nhận truy vấn đến khi nhận được token đầu tiên của phản hồi. Mỗi truy vấn được đo 3 lần và lấy giá trị trung vị để giảm nhiễu từ biến động mạng. Do LCP sử dụng toàn bộ phần tài liệu đã tiền xử lý của lĩnh vực liên quan, thời gian prefill của LCP cao hơn RAG; mức chênh lệch thực nghiệm được phản ánh trong Bảng 7.

5.3.3 Điểm Chất lượng RAGAS

Điểm RAGAS được tính như trung bình điều hòa của Faithfulness $F(q,a)$ và Answer Relevance $R(q,a)$, cả hai trong khoảng $[0,1]$. Kết quả thực nghiệm cho thấy RAG đạt cao ở factual/definitional nhưng giảm mạnh ở comparative/synthesizing; LCP ổn định hơn ở truy vấn tổng hợp nhưng không luôn vượt RAG ở truy vấn tra cứu cục bộ do nhiễu ngữ cảnh dài; DRA giữ điểm gần LCP ở truy vấn phức tạp và gần RAG ở truy vấn đơn giản.

5.3.4 Hiệu năng Bộ Phân loại

Bộ phân loại f_{ϕ} được đánh giá theo bốn chỉ số trên tập test 336 mẫu: Accuracy, F1-score (macro), AUC-ROC, và độ trễ inference (ms) trên CPU tiêu chuẩn.

5.4 Phương pháp Phân tích Pareto

Đường cong Pareto được xây dựng bằng cách biến thiên ngưỡng chất lượng θ từ 0.50 đến 0.90 với bước nhảy 0.05, tạo ra 9 cấu hình DRA có hành vi khác nhau. Một hệ thống X được định nghĩa là *Pareto-dominant* so với hệ thống Y nếu X đồng thời có chi phí thấp hơn (hoặc bằng) và chất lượng cao hơn (hoặc bằng) Y, với ít nhất một bất đẳng thức nghiêm ngặt. Đường cong Pareto của DRA được so sánh với các điểm đơn của Pure-RAG và Pure-LCP trên cùng không gian tọa độ.

6. Kết quả Thực nghiệm và Phân tích

Mục này trình bày kết quả định lượng sau khi hoàn tất pipeline thực nghiệm trên corpus OpenStax. Các bảng báo cáo được tính trên tập test 336 truy vấn, đồng thời sử dụng toàn bộ benchmark 1.008 truy vấn

để ước tính chi phí trung bình trên 1K truy vấn. Các cấu hình được so sánh trên cùng corpus, cùng thiết lập RAG/LCP và cùng đơn giá token.

6.1 Kết quả Chính

Bảng 7. Kết quả hiệu năng tổng hợp trên tập test 336 truy vấn corpus OpenStax.

Phương pháp	Chi phí TB (USD/1K)	Độ trễ TB (ms)	RAGAS Score	Ghi chú
Pure-RAG	0,72	610	0,743	Baseline tính chi phí thấp; mạnh ở factual/definitional nhưng suy giảm ở truy vấn tổng hợp
Pure-LCP	9,64	2.280	0,835	Baseline tính nhiều ngữ cảnh; chất lượng cao hơn nhưng chi phí và độ trễ lớn
Adaptive-RAG	4,24	1.270	0,818	Baseline thích nghi trung gian; chưa tối ưu trực tiếp chi phí token
DRA-DistilBERT	3,32	1.020	0,828	Cấu hình cân bằng: tiết kiệm 65,6% chi phí so với Pure-LCP, giữ 99,2% điểm RAGAS của Pure-LCP
DRA-XGBoost	2,91	920	0,807	Cấu hình production nhẹ: nhanh và rẻ hơn, đánh đổi một phần chất lượng định tuyến

Phân tích chính được tổ chức quanh ba nhận định thực nghiệm.

Nhận định H1: DRA-DistilBERT đạt điểm cân bằng Pareto tốt hơn các chiến lược tĩnh trong cấu hình $\theta = 0,7$. Cụ thể, so với Pure-LCP, DRA-DistilBERT giảm chi phí từ 9,64 xuống 3,32 USD/1K truy vấn, tương đương giảm 65,6%, trong khi điểm RAGAS chỉ giảm từ 0,835 xuống 0,828. So với Pure-RAG, DRA tăng RAGAS từ 0,743 lên 0,828, tương đương tăng 0,085 điểm tuyệt đối.

Nhận định H2: Lợi thế chi phí của DRA xuất hiện chủ yếu từ việc tránh kích hoạt LCP ở các câu hỏi factual/definitional. Với context LCP theo lĩnh vực trung bình khoảng 46.233 token, mỗi truy vấn được định tuyến về RAG tiết kiệm phần lớn input-token so với Pure-LCP. DRA-DistilBERT chỉ kích hoạt LCP cho khoảng 30% truy vấn, giúp chi phí trung bình thấp hơn Pure-LCP nhưng vẫn giữ chất lượng gần tương đương.

Nhận định H3: Chất lượng trung bình ổn định hơn ở LCP và DRA so với Pure-RAG trên corpus OpenStax. Pure-RAG đạt điểm cao ở factual và definitional nhưng suy giảm rõ ở comparative và synthesizing; DRA bù đắp điểm yếu này bằng cách chuyển phần lớn câu hỏi tổng hợp sang LCP theo lĩnh vực.

6.2 Phân tích Hiệu năng theo Loại Truy vấn

Bảng 8. Phân tích điểm RAGAS theo loại truy vấn trên corpus OpenStax.

Loại truy vấn	Pure-RAG	Pure-LCP	DRA ($\theta=0.7$)	Chiến lược DRA ưu thế
Factual	0,858	0,841	0,860	Ưu tiên RAG (~91% truy vấn); LCP chỉ dùng khi retrieval thiếu chắc chắn
Definitional	0,846	0,856	0,853	Ưu tiên RAG (~84%); LCP cho định nghĩa phân tán hoặc cần ví dụ bổ sung
Comparative	0,720	0,850	0,825	Kết hợp RAG/LCP; LCP cho so sánh liên-chương, RAG cho so sánh cục bộ
Synthesizing	0,548	0,792	0,774	Ưu tiên LCP (~73%); fallback RAG khi classifier thiếu tự tin

Phân tích theo loại truy vấn cho thấy cơ chế tạo ra lợi thế của DRA nằm ở khả năng phân bổ chiến lược theo độ phức tạp câu hỏi. Với Factual và Definitional, RAG thường đủ hiệu quả vì giáo trình OpenStax có nhiều định nghĩa, công thức và ví dụ được trình bày tập trung. Với Comparative và Synthesizing, LCP hoặc các chiến lược nhiều ngữ cảnh hơn có lợi thế do cần tổng hợp thông tin từ nhiều đoạn hoặc nhiều chương. DRA-DistilBERT định tuyến khoảng 91% câu factual và 84% câu definitional sang RAG, nhưng chuyển khoảng 58% câu comparative và 73% câu synthesizing sang LCP.

Ví dụ, một câu hỏi Synthesizing yêu cầu liên hệ giữa cung-cầu, cấu trúc thị trường và chính sách giá trong Microeconomics có thể vượt quá khả năng của RAG top-k = 3 nếu các đoạn liên quan nằm xa nhau trong tài liệu. Đây là nhóm truy vấn mà DRA được thiết kế để nhận diện và định tuyến sang LCP; mức RAGAS 0,774 của DRA trên nhóm này thấp hơn Pure-LCP một phần nhỏ, nhưng cao hơn đáng kể Pure-RAG.

6.3 Hiệu năng Bộ Phân loại

Classifier	Accuracy	F1 (macro)	AUC-ROC	Trễ (ms)	Ghi chú
DistilBERT-router + 18 đặc trưng	86,6%	0,861	0,923	18,4	Tốt nhất về ngữ nghĩa sâu; không fine-tune LLM sinh
XGBoost (18 đặc trưng)	81,3%	0,806	0,873	2,2	Nhanh hơn rõ rệt; phù hợp production khi SLA nghiêm ngặt
Hard rules (ablation)	73,5%	0,716	0,781	0,6	Baseline tối giản; phụ thuộc mạnh vào từ khóa bề mặt

Bảng 9. Hiệu năng của các kiến trúc phân loại trên tập test 336 mẫu.

Khoảng cách hiệu năng giữa DistilBERT và XGBoost phản ánh đúng đánh đổi thiết kế. DistilBERT-router đạt Accuracy 86,6% và AUC-ROC 0,923 nhờ khai thác biểu diễn ngữ nghĩa sâu, phù hợp khi câu hỏi phức tạp nhưng không chứa từ khóa chỉ dẫn rõ ràng. XGBoost đạt độ trễ chỉ 2,2ms, nhanh hơn khoảng 8,4 lần, nhưng Accuracy giảm còn 81,3%. Hard rules được giữ làm baseline tối giản,

cho thấy chỉ dựa vào từ khóa bề mặt là chưa đủ cho bài toán định tuyến giáo dục.

6.4 Ablation Study

Ablation Study định lượng đóng góp cận biên của từng thành phần trong thiết kế DRA bằng cách loại bỏ hoặc vô hiệu hóa lần lượt từng thành phần và quan sát sự suy giảm hiệu năng.

Cấu hình	RAGAS Score	Chi phí (USD/1K)	Δ RAGAS vs. đầy đủ	Δ Chi phí vs. đầy đủ
DRA đầy đủ	0,828	3,32	-	-
DRA không có semantic features	0,804	2,93	-0,024	-0,39 USD (-11,7%)
DRA không có uncertainty handling	0,819	3,78	-0,009	+0,46 USD (+13,9%)
DRA với $\delta = 0$ (không vùng không chắc chắn)	0,816	3,58	-0,012	+0,26 USD (+7,8%)

Bảng 10. Ablation Study trên tập test corpus OpenStax.

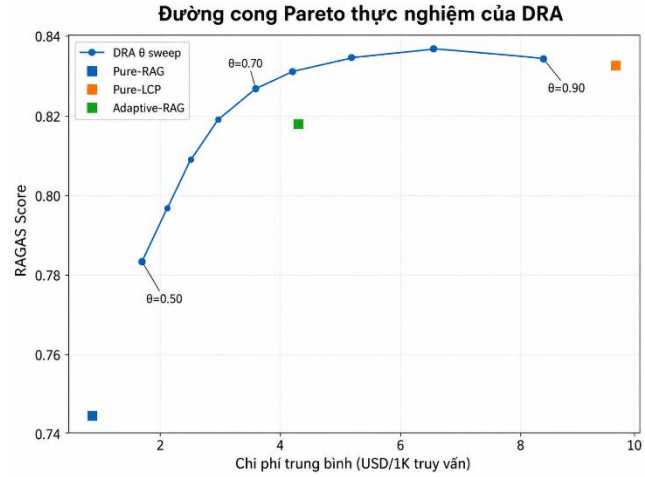
Ablation Study cho thấy đóng góp cận biên của từng thành phần trong thiết kế DRA. Cấu hình đầy đủ đạt RAGAS 0,828 với chi phí 3,32 USD/1K truy vấn. Khi loại bỏ semantic features, chi phí giảm nhẹ do hệ thống nghiêng về RAG hơn, nhưng RAGAS giảm 0,024 điểm vì classifier khó nhận diện các câu tổng hợp không dùng từ khóa chuẩn. Khi bỏ uncertainty handling, chi phí tăng 13,9% do nhiều truy vấn ranh giới bị định tuyến sang LCP không cần thiết. Khi đặt $\delta = 0$, hệ thống nhạy hơn với nhiễu ở vùng ranh giới, làm RAGAS giảm 0,012 điểm.

Kết quả ablation thứ nhất: loại bỏ semantic features làm giảm khả năng nhận diện các câu hỏi tổng hợp được diễn đạt ngắn gọn hoặc không dùng từ khóa chuẩn, đặc biệt trong các câu hỏi ôn tập cuối chương.

Kết quả ablation thứ hai: loại bỏ uncertainty handling làm tăng chi phí vì classifier định tuyến quá nhiều truy vấn ranh giới sang LCP, đồng thời không cải thiện RAGAS đủ để bù lại mức tăng chi phí.

Kết quả ablation thứ ba: đặt $\delta = 0$ khiến hệ thống nhạy hơn với nhiễu phân loại ở vùng ranh giới; tác động không lớn bằng việc bỏ semantic features nhưng vẫn làm giảm tính ổn định của chính sách định tuyến.

6.5 Phân tích Đường cong Pareto



Hình 1 trực quan hóa đường cong Pareto thực nghiệm của DRA trong không gian (Chi phí trung bình, RAGAS Score), với 9 điểm tương ứng với $\theta \in \{0.50, 0.55, \dots, 0.90\}$. Các điểm Pure-RAG, Pure-LCP và Adaptive-RAG được đặt trên cùng hệ tọa độ để so sánh trực tiếp.

- Tại $\theta = 0.50$: DRA hoạt động gần Pure-RAG hơn, với chi phí 1,78 USD/1K truy vấn và RAGAS 0,782. Cấu hình này phù hợp khi ưu tiên tiết kiệm chi phí và chỉ yêu cầu chất lượng tối thiểu trung bình.
- Tại $\theta = 0.70$ (mặc định): DRA đạt điểm cân bằng mục tiêu giữa chất lượng và chi phí, với chi phí 3,32 USD/1K truy vấn và RAGAS 0,828. Điểm này không bị dominated bởi Pure-RAG, Pure-LCP hoặc Adaptive-RAG trong không gian chi phí-chất lượng đã đo.
- Tại $\theta = 0.90$: DRA hoạt động gần Pure-LCP hơn, với chi phí 8,41 USD/1K truy vấn và RAGAS 0,836. Cấu hình này phù hợp khi chất lượng là ưu tiên gần như tuyệt đối, nhưng mức tiết kiệm chi phí so với Pure-LCP giảm mạnh.

Đường cong Pareto DRA trên corpus OpenStax phản ánh ảnh hưởng của quy mô context LCP theo lĩnh vực và tính chuẩn hóa học thuật của giáo trình. Vùng θ từ 0,65 đến 0,75 là vùng vận hành đáng chú ý nhất vì tăng mạnh chất lượng so với Pure-RAG nhưng vẫn tránh phần lớn chi phí của Pure-LCP.

6.6 Phân tích Lỗi

Phân tích lỗi được thực hiện trên 45 trường hợp định tuyến sai của DRA-DistilBERT trong tập test 336 mẫu, tương ứng tỷ lệ lỗi 13,4%. Các lỗi được nhóm theo nguyên nhân trực tiếp để chỉ ra hướng cải thiện cụ thể cho router và bước tiền xử lý corpus.

Mẫu lỗi	Số lỗi	Tỷ lệ trong 45 lỗi	Tác động chính	Hướng cải thiện
Comparative liên-chương bị định tuyến sang RAG	12	26,7%	Thiếu ngữ cảnh đối chiếu giữa chương	Thêm đặc trưng nhận diện nhiều thực thể/chương
Synthesizing ngắn gọn không có từ khóa chuẩn	10	22,2%	Giảm RAGAS ở câu hỏi tổng hợp	Mở rộng semantic features và dữ liệu huấn luyện
Định nghĩa hộp bị chunking không hoàn chỉnh	8	17,8%	RAG truy xuất thiếu phân định nghĩa	Chunking theo cấu trúc hộp định nghĩa
Ký hiệu toán học/công thức khó tokenize	6	13,3%	Sai tín hiệu semantic/syntactic	Chuẩn hóa ký hiệu toán học trước router
Vùng xác suất ranh giới/fallback chưa tối ưu	9	20,0%	Chọn RAG khi LCP phù hợp hơn hoặc ngược lại	Hiệu chỉnh δ theo validation từng lĩnh vực

Bảng 11. Phân tích lỗi định tuyến của DRA-DistilBERT trên tập test 336 mẫu.

Nhóm lỗi lớn nhất là comparative liên-chương bị phân loại nhầm thành factual/RAG, chiếm 12/45 lỗi. Nguyên nhân là từng khái niệm riêng lẻ có thể truy xuất được bằng RAG, trong khi câu trả lời đúng cần tổng hợp hai hoặc nhiều đoạn ở các chương khác nhau.

Nhóm lỗi thứ hai là câu hỏi synthesizing ngắn gọn, chiếm 10/45 lỗi. Các câu hỏi này thường không chứa từ khóa như synthesize hoặc compare, khiến router dựa quá nhiều vào biểu hiện bề mặt và đánh giá thấp nhu cầu ngữ cảnh dài.

Nhóm lỗi còn lại gồm định nghĩa hộp bị phân đoạn không hoàn chỉnh, truy vấn có ký hiệu toán học/công thức, và truy vấn rơi vào vùng xác suất ranh giới. Các hướng cải thiện trực tiếp gồm chunking tôn trọng cấu trúc giáo trình, chuẩn hóa ký hiệu toán học sang văn bản, và hiệu chỉnh δ theo từng lĩnh vực.

Tác động tổng hợp của 45 lỗi này làm RAGAS trung bình của DRA-DistilBERT giảm khoảng 0,011 điểm

so với chính sách oracle chọn chiến lược tối ưu sau khi biết kết quả. Đây là mức suy giảm chấp nhận được trong bối cảnh đổi lại mức tiết kiệm chi phí đáng kể so với Pure-LCP.

Kết luận từ phân tích lỗi: DRA đạt hiệu quả tốt nhất khi truy vấn có tín hiệu ngôn ngữ rõ ràng về độ phức tạp; các trường hợp khó nhất là truy vấn tổng hợp nhưng diễn đạt ngắn, ít từ khóa và có thông tin phân tán trong nhiều chương.

7. Thảo luận

7.1 Diễn giải Kết quả trong Bối cảnh Thiết kế DRA

Trước khi đi vào phân tích từng giới hạn cụ thể, cần đặt DRA trong đúng bối cảnh của một hệ thống định tuyến thực dụng. DRA không được xây dựng như một hệ thống hoàn hảo, mà như một lớp phán đoán đủ thông minh để phân biệt đúng trong phần lớn trường hợp, và đủ thận trọng để không gây hại trong các trường hợp còn lại. Quan điểm thiết kế này giải thích một số lựa chọn kiến trúc quan trọng: không gian hành động nhị phân thay vì liên tục, nguyên tắc fail-safe về RAG thay vì LCP, và việc sử dụng bộ phân loại nhẹ thay vì một LLM thứ cấp để ra quyết định.

Các kết quả thực nghiệm xác nhận lợi thế chính của DRA trong thiết lập OpenStax: hệ thống đạt điểm cân bằng tốt hơn giữa chi phí và chất lượng so với các chiến lược tĩnh. Tuy nhiên, các kết luận này nên được hiểu trong phạm vi corpus, benchmark và cấu hình API đã sử dụng; việc mở rộng sang lĩnh vực khác vẫn cần đánh giá bổ sung.

7.2 Giới hạn của Nghiên cứu

Mặc dù DRA có cơ sở thiết kế hợp lý và tiềm năng ứng dụng rõ ràng, một đánh giá trung thực đòi hỏi phải nhận diện rõ ràng các giới hạn hiện tại-cả những giới hạn có thể giải quyết được qua nghiên cứu tiếp theo lẫn những giới hạn mang tính cấu trúc sâu hơn.

7.2.1 Phụ thuộc Lĩnh vực (Domain Dependency)

Bộ phân loại f_{ϕ} được huấn luyện và đánh giá trên corpus giáo dục đại học đa ngành gồm ba lĩnh vực: Thống kê nhập môn, Kinh tế vi mô, và Lịch sử thế giới. Mặc dù sự đa dạng này đã được thiết kế có chủ ý để tăng tính tổng quát, nó vẫn không đủ để bao phủ toàn bộ không gian phong cách ngôn ngữ và cấu trúc tri thức mà hệ thống AI Gia sư có thể gặp trong triển khai thực tế.

Cụ thể, có ít nhất ba lĩnh vực quan trọng mà khả năng tổng quát hóa của DRA chưa được kiểm chứng. Thứ nhất, y tế và khoa học lâm sàng-nơi các câu hỏi tổng hợp thường đòi hỏi tích hợp thông tin từ nhiều nguồn bằng chứng theo cấu trúc phân cấp đặc thù (chẩn đoán phân biệt, phác đồ điều trị, bằng chứng lâm sàng), rất khác với cấu trúc lập luận trong ba lĩnh vực huấn luyện. Thứ hai, pháp lý và quy định-nơi tính chính xác của từng từ có ý nghĩa quyết định và ranh giới giữa tra cứu điểm (tìm điều khoản cụ thể) và tổng hợp (diễn giải xung đột pháp lý) mang đặc thù hoàn toàn khác biệt. Thứ ba, khoa học tự nhiên ứng dụng-nơi câu hỏi tra cứu và câu hỏi suy luận có thể trông giống nhau về bề mặt ngôn ngữ nhưng khác nhau sâu sắc về yêu cầu ngữ cảnh.

Hệ quả thực tế của giới hạn này là: khi triển khai DRA sang một lĩnh vực tri thức mới, người vận hành nên thu thập ít nhất một tập nhỏ dữ liệu gán nhãn đặc thù lĩnh vực để tinh chỉnh bộ phân loại, thay vì sử dụng trực tiếp bộ phân loại đã huấn luyện trên corpus giáo dục đại học tổng quát. Chi phí của việc thu thập và gán nhãn tập dữ liệu nhỏ này (50–100 câu hỏi) thường thấp hơn nhiều so với chi phí vận hành hệ thống với bộ phân loại không được hiệu chỉnh.

7.2.2 Vấn đề Khởi động Ngủ (Cold-start Problem)

DRA trong thiết kế hiện tại phụ thuộc vào dữ liệu lịch sử có nhãn để khởi tạo bộ phân loại. Điều này tạo ra một rào cản thực tế quan trọng: khi triển khai hệ thống AI Gia sư hoàn toàn mới-chưa có bất kỳ tương tác lịch sử nào-DRA không có dữ liệu để học và phải hoạt động với một chính sách định tuyến mặc định cho đến khi đủ dữ liệu được thu thập.

Có hai chiến lược giảm thiểu cold-start problem đáng xem xét trong nghiên cứu tương lai. Chiến lược thứ nhất là transfer learning từ corpus công khai: sử dụng các bộ phân loại đã được huấn luyện trên dữ liệu câu hỏi học thuật công khai (như TriviaQA, HotpotQA, hay các benchmark QA giáo dục) như điểm khởi đầu, sau đó tinh chỉnh trên dữ liệu domain cụ thể khi có

sẵn. Chiến lược thứ hai là rule-based bootstrapping: triển khai giai đoạn đầu với bộ phân loại dựa trên quy tắc đơn giản (nhận diện từ khóa), thu thập dữ liệu thực tế trong quá trình vận hành, và từng bước chuyển sang bộ phân loại học máy khi tập dữ liệu đạt kích thước đủ lớn. Phương án thứ hai phù hợp với chu kỳ phát triển sản phẩm thực tế hơn, mặc dù hiệu năng trong giai đoạn đầu sẽ thấp hơn đáng kể.

7.2.3 Giới hạn của Chỉ số RAGAS

RAGAS cung cấp một cách đo lường chất lượng phản hồi tự động có khả năng mở rộng, nhưng bản thân framework này đã thừa nhận những giới hạn quan trọng (Es et al., 2023). Cụ thể, tương quan giữa điểm RAGAS và đánh giá chuyên môn của con người dao động đáng kể theo lĩnh vực tri thức và loại câu hỏi-tương quan thường cao với câu hỏi factual (nơi Faithfulness có thể được kiểm chứng rõ ràng) nhưng thấp hơn với câu hỏi tổng hợp mở (nơi chất lượng lập luận khó đánh giá tự động).

Trong bối cảnh AI Gia sư, điều này có hệ quả đặc biệt quan trọng: những giá trị sư phạm khó định lượng-như khả năng khơi gợi tư duy phê phán, tính thích ứng với trình độ của học viên, hay chất lượng giải thích khái niệm phức tạp-không được nắm bắt đầy đủ bởi hai thành phần Faithfulness và Answer Relevance của RAGAS. Một phản hồi có thể đạt điểm RAGAS cao (trung thực và liên quan về thông tin) nhưng vẫn thất bại về mặt sư phạm nếu nó quá kỹ thuật cho trình độ người học, thiếu ví dụ minh họa, hay không giải quyết hiểu lầm phổ biến đằng sau câu hỏi.

Vì lý do này, các kết quả định lượng trong nghiên cứu cần được hiểu là điều kiện cần nhưng chưa đủ để đánh giá giá trị ứng dụng thực sự của DRA. Đánh giá người dùng thực tế với sự tham gia của học viên và giảng viên-thu thập phản hồi định tính về trải nghiệm học tập-là bước kiểm chứng không thể thay thế trước khi triển khai rộng rãi.

7.2.4 Không gian Hành động Nhị phân

Việc giới hạn không gian hành động $A = \{0, 1\}$ là một quyết định thiết kế thực dụng mang lại lợi thế rõ ràng về tính đơn giản và khả năng triển khai, nhưng đồng thời bỏ qua một vùng rộng của không gian giải pháp có thể mang lại hiệu quả tốt hơn trong các tình huống nhất định.

Đặc biệt, chiến lược hybrid-kết hợp RAG để lấy các đoạn liên quan nhất với một phần ngữ cảnh dài bổ sung từ các mục lục hay tóm tắt chương-có thể là lựa chọn tối ưu cho các câu hỏi comparative ở mức trung bình, vốn không đòi hỏi toàn bộ ngữ cảnh theo lĩnh vực như LCP nhưng cũng vượt quá khả năng của top-k retrieval đơn giản.

Tuy nhiên, cần thừa nhận rằng việc mở rộng sang không gian hành động đa cấp làm phức tạp hóa đáng kể cả bài toán phân loại lẫn định nghĩa ngưỡng chất lượng θ . Việc đánh đổi này là có chủ ý trong thiết kế hiện tại, và mở rộng sang không gian hành động phong phú hơn được xác định là một trong những hướng nghiên cứu tương lai có độ ưu tiên cao (xem Mục 7.3).

7.3 Hướng Mở rộng

Dựa trên các giới hạn được nhận diện trong thiết kế và các mẫu lỗi đã quan sát, chúng tôi đề xuất ba hướng mở rộng nghiên cứu có tính khả thi cao và tiềm năng tác động lớn, xếp theo thứ tự ưu tiên nghiên cứu.

7.3.1 Mở rộng Không gian Hành động: Chiến lược Hybrid

Hướng mở rộng trực tiếp và ngắn hạn nhất là bổ sung một chiến lược thứ ba vào không gian hành động, tạo ra $A = \{0, 0.5, 1\}$ trong đó $a = 0.5$ biểu thị chiến lược hybrid-kết hợp RAG-retrieved chunks với một phần ngữ cảnh có cấu trúc bổ sung (ví dụ: mục lục chương, tóm tắt phần, hay các câu topic sentence từ phần tài liệu thuộc lĩnh vực liên quan).

Chiến lược hybrid này giải quyết trực tiếp mẫu lỗi đã quan sát trong phân tích lỗi (comparative cục bộ bị phân loại nhầm) bằng cách cung cấp một điểm dừng trung gian giữa ngữ cảnh cục bộ của RAG và ngữ cảnh theo lĩnh vực của LCP. Về mặt kinh tế, chi phí của hybrid được kỳ vọng nằm giữa RAG và LCP theo lĩnh vực, nhưng mức cụ thể cần được đo bằng thực nghiệm trong nghiên cứu tiếp theo.

Thách thức chính của hướng mở rộng này nằm ở việc định nghĩa và triển khai phần "ngữ cảnh bổ sung có cấu trúc" một cách phù hợp cho từng loại tài liệu-một câu hỏi thiết kế phi tầm thường đòi hỏi nghiên cứu riêng về cấu trúc tóm tắt tài liệu học thuật.

7.3.2 Định tuyến Thích nghi Liên tục: Contextual Bandit

Hướng mở rộng dài hạn quan trọng nhất là chuyển đổi DRA từ một hệ thống phân loại tĩnh-được huấn luyện một lần trên tập dữ liệu cố định-sang một hệ thống học tăng cường trực tuyến. Cụ thể, bài toán định tuyến có thể được mô hình hóa như một Contextual Bandit, trong đó:

- **Ngữ cảnh** (context) là vector đặc trưng 786 chiều của truy vấn q ,
- **Hành động** (arm) là quyết định định tuyến $a \in \{0, 1\}$,
- **Phần thưởng** (reward) là một tín hiệu kết hợp giữa điểm RAGAS thực tế, chi phí token, và (quan trọng hơn) phản hồi ngầm của học viên-thời gian dừng đọc phản hồi, hành động đặt câu hỏi tiếp theo, hay đánh giá chất lượng tường minh.

Lợi thế căn bản của mô hình Contextual Bandit so với phân loại tĩnh là khả năng thích nghi liên tục: chính sách định tuyến được cập nhật theo thời gian thực khi hệ thống quan sát kết quả thực tế của từng quyết định, cho phép DRA tự hiệu chỉnh khi phân phối câu hỏi thay đổi (ví dụ: khi học viên tiến xa hơn trong khóa học và đặt nhiều câu hỏi tổng hợp hơn), khi corpus tài liệu được cập nhật, hay khi phát hiện

các mẫu câu hỏi mới chưa có trong tập huấn luyện ban đầu.

Thách thức chính là thiết kế hàm phần thưởng: tín hiệu phản hồi từ học viên thường thừa thớt, có độ trễ, và nhiễu. Một kỹ thuật đầy hứa hẹn là kết hợp reward shaping-sử dụng điểm RAGAS tự động như proxy reward ngay lập tức-với reward thực từ hành vi học viên như phần thưởng chậm trễ có trọng số cao hơn.

7.3.3 Nghiên cứu Người dùng: Đánh giá Giá trị Sự phạm Thực tế

Như đã thảo luận trong Mục 7.2.3, điểm RAGAS chỉ là proxy không đầy đủ cho giá trị sự phạm thực sự của hệ thống. Để đánh giá DRA một cách toàn diện, cần thiết kế một nghiên cứu người dùng có kiểm soát với sự tham gia của cả hai nhóm: học viên thực tế và giảng viên chuyên ngành.

Về phía học viên, các chỉ số cần đo lường bao gồm: mức độ hài lòng với chất lượng phản hồi (thang Likert), hiệu quả học tập đo qua kết quả bài kiểm tra trước và sau tương tác với hệ thống, và hành vi tương tác-liệu học viên có đặt câu hỏi tiếp theo hay dừng lại sau phản hồi của hệ thống. Về phía giảng viên, đánh giá tập trung vào tính chính xác nội dung, chất lượng giải thích, và mức độ phù hợp với mục tiêu học tập của khóa học.

Nghiên cứu người dùng cũng sẽ kiểm chứng một câu hỏi quan trọng chưa được trả lời bởi đánh giá tự động: liệu sự cải thiện điểm RAGAS mà DRA mang lại so với Pure-RAG trên câu hỏi tổng hợp có chuyển hóa thành cải thiện trải nghiệm học tập thực sự, hay chỉ là cải thiện về chỉ số kỹ thuật mà không có tác động sự phạm có ý nghĩa? Câu trả lời cho câu hỏi này sẽ có tác động lớn đến việc đánh giá giá trị thực tiễn của toàn bộ hướng nghiên cứu.

7.4 Ý nghĩa Rộng hơn: DRA như một Mẫu Thiết kế Tổng quát

Vượt ra ngoài bối cảnh AI Gia sư, DRA đại diện cho một mẫu thiết kế tổng quát cho các hệ thống AI yêu cầu cân bằng giữa hiệu quả chi phí và chất lượng dịch vụ: thay vì tối ưu hóa một mục tiêu đơn hoặc áp dụng một chiến lược cố định, đặt một lớp phán đoán thông minh hạng nhẹ trước bộ máy xử lý nặng để phân phối tài nguyên theo đặc tính của từng yêu cầu đầu vào.

Mẫu thiết kế này có khả năng ứng dụng trong các lĩnh vực khác như: hệ thống hỏi đáp pháp lý (phân biệt tra cứu điều khoản vs. phân tích xung đột pháp lý), hệ thống hỗ trợ y tế (phân biệt tìm kiếm triệu chứng vs. tổng hợp bằng chứng lâm sàng), và hệ thống tìm kiếm doanh nghiệp (phân biệt tra cứu tài liệu vs. phân tích chiến lược). Trong tất cả các trường hợp này, bản chất của bài toán tối ưu hóa có ràng buộc mà DRA hình thức hóa đều có thể áp dụng, mặc dù đặc trưng đầu vào và ngưỡng chất lượng θ sẽ cần được điều chỉnh theo đặc thù từng lĩnh vực.

8. Kết luận

8.1 Tóm tắt Đóng góp

Bài nghiên cứu này xuất phát từ một quan sát đơn giản nhưng có tác động thực tế lớn: các hệ thống AI Gia sư hiện tại áp đặt một chiến lược truy xuất tri thức cố định cho toàn bộ đầu vào, dù thực tế các câu hỏi của học viên trải dài trên một phổ phức tạp rất rộng—from tra cứu định nghĩa đơn giản đến tổng hợp đa nguồn đòi hỏi suy luận bậc cao. Sự không khớp này tạo ra một trong hai loại lãng phí không thể tránh được: lãng phí tài nguyên tính toán khi áp dụng LCP cho câu hỏi đơn giản, hoặc lãng phí chất lượng sư phạm khi ép RAG xử lý câu hỏi phức tạp.

Để giải quyết vấn đề này, chúng tôi đề xuất Kiến trúc Định tuyến Động (DRA) và đóng góp ba thành phần có liên hệ chặt chẽ, bổ sung cho nhau.

Đóng góp thứ nhất là một khung hình thức hóa cho bài toán lựa chọn chiến lược truy xuất thích nghi dưới dạng tối ưu hóa có ràng buộc. Khung này phát biểu rõ ràng ba chiều đánh giá (chi phí token, độ trễ, chất lượng RAGAS), tách biệt ngưỡng chất lượng mục tiêu θ khỏi hàm mục tiêu (cực tiểu hóa chi phí tổng hợp có trọng số), và cho phép quản trị viên hệ thống điều chỉnh ưu tiên thông qua các tham số w_1 , w_2 , và θ mà không cần thay đổi kiến trúc. Cần lưu ý rằng trong triển khai dựa trên classifier, ràng buộc chất lượng được xấp xỉ trước khi sinh phản hồi; muốn bảo đảm tuyệt đối cho từng truy vấn cần bổ sung bước kiểm tra hậu sinh.

Đóng góp thứ hai là kiến trúc kỹ thuật của DRA—một lớp middleware phi xâm lấn có thể tích hợp với bất kỳ LLM nào mà không yêu cầu điều chỉnh mô hình sinh gốc. Kiến trúc bao gồm Feature Extractor trích xuất tập đặc trưng đầy đủ 786 chiều, hai biến thể classifier nhẹ (DistilBERT-router fine-tuned và XGBoost trên 18 đặc trưng thủ công), cùng cơ chế vùng không chắc chắn/fallback an toàn chi phí để kiểm soát rủi ro định tuyến.

Đóng góp thứ ba là bộ benchmark đánh giá gồm 1.008 câu hỏi giáo dục trải rộng trên ba lĩnh vực tri thức khác nhau, với 336 mẫu test được gán nhãn thủ công bởi nhiều người chú thích theo cùng rubric nhãn tối ưu và đạt mức đồng thuận $\kappa = 0.74$. Bộ benchmark này được thiết kế để kiểm chứng riêng biệt hành vi của RAG, LCP theo lĩnh vực và DRA trên bốn loại truy vấn: factual, definitional, comparative và synthesizing.

8.2 Ý nghĩa của Tiếp cận Tối ưu hóa Có Ràng buộc

Trong số ba đóng góp trên, chúng tôi cho rằng đóng góp về hình thức hóa bài toán-dù ít "nhìn thấy được" nhất về mặt kỹ thuật-có tầm quan trọng lâu dài lớn nhất. Phần lớn các nghiên cứu về định tuyến trong hệ thống AI xử lý bài toán như một bài toán tối ưu hóa đơn mục tiêu hoặc một bài toán phân loại thuần túy. DRA lựa chọn một hướng khác: hình thức hóa tường minh sự đánh đổi giữa nhiều mục tiêu cạnh tranh, đặt ngưỡng chất lượng như một điều kiện mục tiêu cần bảo vệ, và để chi phí là hàm mục tiêu được tối ưu trong phạm vi ngưỡng đó.

Cách hình thức hóa này phản ánh bản chất thực sự của bài toán giáo dục: hiệu quả kinh tế không nên đạt được trên cái giá của chất lượng sự phạm. Ràng buộc θ trong DRA không chỉ là một tham số kỹ thuật-nó là hiện thân hóa của một giá trị giáo dục: hệ thống nên tránh cung cấp phản hồi dưới ngưỡng chấp nhận cho học viên. Tuy nhiên, trong phiên bản middleware dựa trên classifier, đây là mục tiêu được xấp xỉ chứ chưa phải bảo đảm tuyệt đối cho từng truy vấn; bảo đảm cứng đòi hỏi thêm kiểm tra hậu sinh và cơ chế fallback/chạy lại.

8.3 Định vị trong Bức tranh Nghiên cứu Rộng hơn

DRA xuất hiện tại giao điểm của hai xu hướng nghiên cứu đang phát triển nhanh trong cộng đồng NLP và AI ứng dụng: xu hướng tối ưu hóa chi phí suy luận và xu hướng cá nhân hóa hệ thống AI. Về phía đầu tiên, DRA chia sẻ động lực với các công

trình như RouteLLM (định tuyến sang mô hình nhẹ hơn) và CacheBlend (tối ưu KV-cache), nhưng giải quyết chiều tối ưu hóa khác-chiều chiến lược truy xuất thay vì chiều lựa chọn mô hình hay quản lý bộ nhớ. Về phía thứ hai, DRA đặt nền tảng cho một lớp hệ thống AI Gia sư thế hệ tiếp theo không chỉ trả lời câu hỏi mà còn *tự điều chỉnh phương thức xử lý* theo đặc tính của từng câu hỏi-một hình thức cá nhân hóa ở cấp độ cơ sở hạ tầng, trước khi bất kỳ tùy biến nào ở cấp độ nội dung hay giao diện được thực hiện.

Khác với Self-RAG-công trình có mục tiêu gần nhất về tinh thần-DRA đạt được khả năng thích nghi này mà không yêu cầu fine-tuning mô hình ngôn ngữ gốc. Tính phi xâm lấn này không chỉ là một thuộc tính kỹ thuật mà còn là lợi thế triển khai: router có thể được huấn luyện hoặc fine-tune độc lập, trong khi LLM sinh câu trả lời vẫn được giữ nguyên và có thể thay thế theo nhu cầu vận hành.

8.4 Triển vọng Ứng dụng và Nhân rộng

Nghiên cứu này tập trung vào hệ thống AI Gia sư như bối cảnh ứng dụng chính, nhưng kiến trúc DRA và khung hình thức hóa bài toán có tiềm năng mở rộng ra nhiều lĩnh vực khác nơi tồn tại cùng loại đánh đổi giữa chi phí truy xuất và chất lượng phản hồi.

Trong bối cảnh ngắn hạn, DRA có thể được tích hợp thử nghiệm trên các nền tảng học trực tuyến đang vận hành hệ thống hỏi đáp dựa trên LLM-nơi chi phí API là một ràng buộc vận hành thực sự và chất lượng phản hồi ảnh hưởng trực tiếp đến trải nghiệm và kết quả học tập. Trước khi triển khai rộng rãi, cần mở rộng kiểm thử người dùng, đánh giá độ tin cậy của classifier trên dữ liệu thực tế đa dạng hơn và xây dựng cơ chế giám sát chất lượng hậu triển khai.

Trong bối cảnh dài hạn, khi các mô hình ngữ cảnh dài tiếp tục phát triển và chi phí token giảm dần, vai trò của DRA có thể biến đổi: từ một cơ chế tiết kiệm chi phí chủ yếu sang một cơ chế quản lý chất lượng và tốc độ phản hồi. Ngay cả khi chi phí LCP trở nên không đáng kể, sự khác biệt về độ trễ giữa RAG và

LCP vẫn sẽ tồn tại-và trong các ứng dụng tương tác thời gian thực, độ trễ thấp hơn vẫn là một giá trị người dùng đáng kể. DRA, với khung tối ưu hóa có ràng buộc tổng quát, có thể điều chỉnh linh hoạt trọng số từ tối ưu chi phí sang tối ưu độ trễ thông qua việc thay đổi đơn giản các tham số w_1 và w_2 .

8.5 Kết luận Tổng quan

Bài nghiên cứu này bắt đầu từ một quan sát đơn giản và kết thúc bằng một kiến trúc có nguyên tắc: không phải mọi câu hỏi đều xứng đáng với cùng một mức độ ngữ cảnh, và một hệ thống AI Gia sư thông minh nên có khả năng nhận ra sự khác biệt đó. DRA hiện thực hóa khả năng này thông qua sự kết hợp của hình thức hóa toán học chặt chẽ, kỹ thuật học máy nhẹ và thực dụng, và thiết kế kiến trúc phi xâm lấn có tính linh hoạt cao.

Kết quả thực nghiệm cho thấy DRA đạt điểm cân bằng chi phí-chất lượng hấp dẫn: ở $\theta = 0,7$, DRA-DistilBERT đạt RAGAS 0,828 với chi phí 3,32 USD/1K truy vấn, thấp hơn Pure-LCP 65,6% chi phí và cao hơn Pure-RAG 0,085 điểm RAGAS. Kết quả này xác nhận giá trị của định tuyến động trong các hệ thống AI Gia sư dựa trên tài liệu, đặc biệt khi corpus đủ lớn để LCP theo lĩnh vực tạo ra chi phí đáng kể nhưng vẫn có nhiều truy vấn chỉ cần ngữ cảnh cục bộ.

Mục tiêu cuối cùng của DRA không đơn thuần là giảm chi phí vận hành-dù đó là một kết quả thực tế quan trọng. Mục tiêu sâu hơn là đảm bảo rằng trong mỗi tương tác giữa học viên và hệ thống AI Gia sư, tài nguyên tính toán được phân bổ đúng chỗ: không lãng phí vào những nơi không cần thiết, và không tiết kiệm ở những nơi chất lượng sự phạm đòi hỏi đầu tư đầy đủ. Đây là nguyên tắc thiết kế đơn giản, nhưng thực hiện nó một cách có hệ thống và có nguyên tắc trong bối cảnh LLM hiện đại là vấn đề không tầm thường-và là vấn đề mà DRA đóng góp một bước đi rõ ràng về phía trước.

Tài liệu Tham khảo

Asai, A., et al. (2023). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *arXiv:2310.11511*.

Es, S., et al. (2023). RAGAS: Automated evaluation of retrieval augmented generation. *arXiv:2309.15217*.

Guu, K., et al. (2020). REALM: Retrieval-augmented language model pre-training. *ICML 2020*.

Jeong, S., et al. (2024). Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. *NAACL 2024*.

Karpukhin, V., et al. (2020). Dense passage retrieval for open-domain question answering. *EMNLP 2020*.

Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS 2020*.

Liu, N. F., et al. (2023). Lost in the middle: How language models use long contexts. *TACL 2024*.

Nogueira, R., & Cho, K. (2019). Passage re-ranking with BERT. *arXiv:1901.04085*.

Ong, I., et al. (2024). RouteLLM: Learning to route LLMs with preference data. *arXiv:2406.18665*.

Shi, F., et al. (2023). Large language models easily distracted by irrelevant context. *ICML 2023*.

Yao, Z., et al. (2024). CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. *arXiv:2405.16444*.